

Cuprins

Lista figurilor	3
Lista tabelelor	4
Lista acronimelor	5
Introducere	6
1. Noțiuni teoretice	8
1.1. Accelerometre MEMS și principiul de funcționare	8
1.2. Interfața I ² C	9
1.3. Sistemul Linux și modelul driverelor de dispozitiv	10
1.4. Subsistemul Industrial I/O (IIO)	11
1.5. Device Tree în Linux	12
2. Platforma hardware și software	14
2.1. Platforma hardware utilizată	15
2.2. Mediul software utilizat	15
2.3. Conectarea senzorului și verificarea comunicației I2C	17
2.4. Declararea senzorului și a întreruperii prin Device Tree	18
3. Proiectarea și implementarea driverului IIO	21
3.1. Analiza registrelor relevante ale senzorului MMA8452Q	21
3.2. Structura driverului Linux implementat	22
3.3. Inițializarea și identificarea dispozitivului	23
3.4. Configurarea senzorului	24
3.5. Citirea datelor și conversia valorilor brute	25
3.6. Integrarea cu subsistemul IIO	26
3.7. Implementarea întreruperilor hardware	28
3.8. Implementarea triggerului și bufferului IIO	29
3.9. Eliberarea resurselor la eliminarea driverului	31
3.10. Concluzii privind implementarea driverului	32

4. Configurare și integrare prin Device Tree	33
4.1. Descrierea dispozitivului în Device Tree	33
4.2. Configurarea întreruperii în Device Tree	35
4.3. Asocierea driverului cu dispozitivul	36
4.4. Încărcarea și verificarea driverului în sistem	37
5. Aplicația user-space și validare experimentală	40
5.1. Interfața user-space pentru accesarea datelor IIO	41
5.2. Citirea valorilor în mod direct prin sysfs	41
5.3. Citirea datelor prin bufferul IIO	42
5.4. Configurarea domeniului de măsurare și a frecvenței	43
5.5. Calibrare de bază și prelucrarea valorilor	44
5.6. Testarea funcționării driverului	44
5.7. Validarea experimentală a rezultatelor	45
5.8. Impactul asupra resurselor și limitele implementării	46
Concluzii	48
Analiza rezultatelor obținute	48
Opinia personală	49
Limitele implementării	50
Perspective viitoare de dezvoltare	50
Bibliografie	52
Anexă 1 - Codul sursă	54
.1. Driver Linux IIO pentru MMA8452Q	54
.2. Aplicația user-space Python	62
.3. Overlay Device Tree pentru MMA8452Q	74
.4. Makefile pentru compilarea modulului kernel	74

Lista figurilor

1.1.	Schema simplificată a unui accelerometru MEMS capacitiv	8
1.2.	Fluxul de achiziție a datelor folosind întrerupere hardware, trigger IIO și buffer .	12
1.3.	Fluxul de asociere a dispozitivului descris în Device Tree cu driverul I2C și înregistrarea acestuia în subsistemul IIO	13
2.1.	Platforma experimentală utilizată: Raspberry Pi 4 Model B și modulul MMA8452Q conectat prin I2C	14
3.1.	Schema logică de funcționare a driverului MMA8452Q	23
3.2.	Atributele sysfs expuse de driverul IIO pentru accelerometrul MMA8452Q . . .	28
3.3.	Corelarea întreruperii hardware cu triggerul IIO	29
4.1.	Corelarea întreruperii hardware cu triggerul IIO în <code>/proc/interrupts</code>	35
4.2.	Citirea datelor brute din bufferul IIO	39
5.1.	Interfața grafică a aplicației user-space pentru testarea accelerometrului	40
5.2.	Aplicația user-space în modul de achiziție prin buffer IIO	43

Lista tabelelor

2.1.	Conectarea accelerometrului MMA8452Q la Raspberry Pi 4 Model B	15
3.1.	Registre MMA8452Q utilizate în driver	22
5.1.	Structura unui eşantion citit din bufferul IIO	42
5.2.	Configurarea domeniului de măsurare din aplicație	43

Lista acronimelor

ADC (Analog to Digital Converter) = Convertor Analogic-Digital
API (Application Programming Interface) = Interfață de Programare a Aplicațiilor
ARM (Advanced RISC Machine) = Arhitectură de Procesor RISC
CPU (Central Processing Unit) = Unitate Centrală de Procesare
CSV (Comma-Separated Values) = Valori Separate prin Virgulă
DRDY (Data Ready) = Date Disponibile
DTC (Device Tree Compiler) = Compilator pentru Device Tree
DTB (Device Tree Blob) = Fișier Binar Device Tree
DTBO (Device Tree Blob Overlay) = Fișier Binar Overlay Device Tree
DTS (Device Tree Source) = Fișier Sursă Device Tree
GPIO (General Purpose Input Output) = Intrări/Ieșiri Generale
GPL (General Public License) = Licență Publică Generală
GUI (Graphical User Interface) = Interfață Grafică cu Utilizatorul
I2C (Inter-Integrated Circuit) = Magistrală Serială Inter-Integrată
IIO (Industrial Input Output) = Subsistem Industrial de Intrare/Ieșire
IRQ (Interrupt Request) = Cerere de Întrerupere
LSB (Least Significant Bit) = Bitul Cel Mai Puțin Semnificativ
MEMS (Micro-Electro-Mechanical Systems) = Sisteme Micro-Electro-Mecanice
MSB (Most Significant Bit) = Bitul Cel Mai Semnificativ
ODR (Output Data Rate) = Rată de Ieșire a Datelor
OS (Operating System) = Sistem de Operare
SCL (Serial Clock Line) = Linie Serială de Ceas
SDA (Serial Data Line) = Linie Serială de Date
SMBus (System Management Bus) = Magistrală de Management al Sistemului
SoC (System on Chip) = Sistem pe Cip
sysfs (System Filesystem) = Sistem Virtual de Fișiere pentru Obiecte Kernel
UART (Universal Asynchronous Receiver Transmitter) = Transmițător-Receptor Asincron Universal
WHO_AM_I (Who Am I Register) = Registru de Identificare al Senzorului

Introducere

Introducere

Sistemele embedded folosesc tot mai des senzori pentru măsurarea și monitorizarea mediului fizic. Din acest motiv, integrarea dispozitivelor de măsură în Linux a devenit importantă pentru aplicațiile moderne.

Accelerometrele sunt utilizate în electronică embedded, Internet of Things, monitorizarea mișcării, detecția vibrațiilor și orientarea spațială. Realizarea unui driver Linux pentru un astfel de senzor permite studierea legăturii dintre hardware, kernel și aplicațiile din spațiul utilizatorului.

Prezenta lucrare are ca obiect proiectarea și implementarea unui driver Linux, în limbajul C, pentru accelerometrul triaxial MMA8452Q produs de NXP, conectat prin magistrala I²C la o platformă Raspberry Pi 4 Model B care rulează sistemul de operare Armbian. Driverul este integrat în subsistemul Industrial I/O (IIO) al nucleului Linux, astfel încât datele furnizate de senzor să poată fi accesate printr-o interfață standardizată[1].

Motivația alegerii temei

Alegerea accelerometrului MMA8452Q a fost determinată de caracteristicile sale tehnice și de relevanța sa pentru tema abordată. Acesta permite măsurarea accelerației pe trei axe, comunică prin interfața I²C și oferă posibilitatea configurării unor parametri importanți, precum domeniul de măsură și rata de eșantionare. În plus, structura sa internă bazată pe registre îl face potrivit pentru evidențierea etapelor specifice dezvoltării unui driver: identificarea dispozitivului, inițializarea, configurarea și citirea datelor.

Scopul lucrării

În cadrul lucrării se urmărește implementarea unui driver funcțional pentru MMA8452Q, capabil să verifice prezența dispozitivului prin citirea registrului `WHO_AM_I`, să inițializeze senzorul, să permită configurarea parametrilor de funcționare și să furnizeze valorile accelerației pentru axele X, Y și Z. De asemenea, se urmărește integrarea dispozitivului prin Device Tree, astfel încât asocierea driverului cu senzorul să se realizeze automat la pornirea sistemului. Pentru validarea experimentală a soluției propuse este realizată și o aplicație user-space, utilizată pentru citirea datelor expuse de driver, aplicarea unei calibrări de bază și analiza rezultatelor obținute.

Obiectivele principale ale lucrării sunt:

- studierea principiilor de funcționare ale accelerometrelor și a modului de integrare a acestora în Linux prin subsistemul Industrial I/O;
- analizarea interfeței I²C și a mecanismului de comunicare cu senzorul MMA8452Q;

- proiectarea și implementarea în limbajul C a unui driver Linux pentru accelerometrul MMA8452Q;
- integrarea dispozitivului în sistem prin Device Tree;
- testarea funcționării driverului prin citirea registrelor relevante și validarea experimentală a datelor furnizate;
- realizarea unei aplicații user-space pentru demonstrarea funcționării, citirea datelor și aplicarea unei calibrări de bază.

Lucrarea evidențiază etapele practice necesare dezvoltării unui driver Linux pentru un senzor conectat prin I²C, de la configurarea hardware și integrarea în nucleu până la expunerea datelor într-o formă standardizată către aplicațiile din spațiul utilizatorului.

Capitolul 1

Noțiuni teoretice

1.1 Accelerometre MEMS și principiul de funcționare

Accelerometrele sunt senzori utilizați pentru măsurarea accelerației pe una sau mai multe axe. În aplicațiile embedded moderne, cele mai răspândite accelerometre sunt cele realizate în tehnologie MEMS, deoarece au dimensiuni reduse, consum redus de energie și pot fi integrate ușor în sisteme digitale. Un accelerometru MEMS transformă mișcarea mecanică a unei structuri microscopice interne într-un semnal electric care poate fi ulterior prelucrat de circuitele integrate ale sensorului. În funcție de orientarea sensorului și de mișcarea sistemului, valorile măsurate pe axele X, Y și Z se modifică, permițând determinarea poziției, vibrațiilor sau accelerațiilor dinamice.

În cazul accelerometrelor capacitive MEMS, principiul de funcționare se bazează pe deplasarea unei mase microscopice suspendate. Atunci când asupra sensorului acționează o accelerație, masa internă se deplasează față de poziția de echilibru, modificând capacitățile electrice formate între elementele fixe și cele mobile. Această variație de capacitate este convertită de circuitele interne ale sensorului într-o valoare numerică. Astfel, accelerația mecanică este transformată într-o mărime digitală care poate fi citită de un microcontroler, procesor sau sistem embedded.

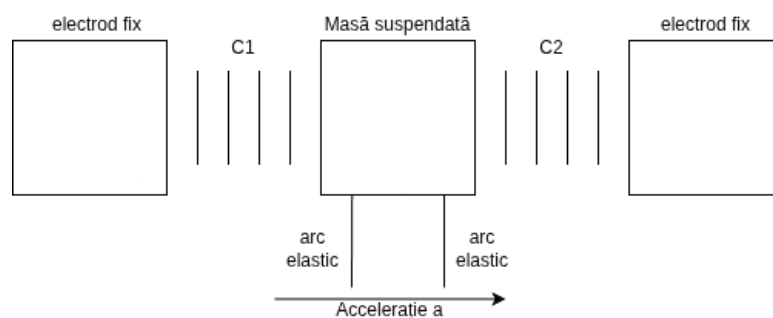


Figura 1.1: Schema simplificată a unui accelerometru MEMS capacitiv

Accelerometrul utilizat în această lucrare este MMA8452Q, produs de NXP Semiconductors. Acesta este un accelerometru triaxial cu ieșire digitală, rezoluție de 12 biți și interfață I2C. Sensorul permite măsurarea accelerației pe cele trei axe spațiale și oferă mai multe domenii de măsurare configurabile: $\pm 2g$, $\pm 4g$ și $\pm 8g$. Alegerea domeniului influențează sensibilitatea măsurării: un domeniu mai mic oferă o rezoluție mai bună pentru accelerații reduse, în timp ce un domeniu mai mare permite măsurarea unor accelerații mai ridicate fără saturarea ieșirii [2, 3]. Un aspect important în utilizarea unui accelerometru digital este conversia valorilor brute în

unități fizice. Valorile citite din registre reprezintă numere întregi codificate pe un anumit număr de biți, iar interpretarea lor depinde de domeniul de măsurare selectat. Pentru MMA8452Q, în modul de 12 biți, valorile brute trebuie extinse cu semn și apoi înmulțite cu factorul de scalare corespunzător domeniului selectat. În implementarea realizată, factorii de scalare utilizați sunt asociați domeniilor $\pm 2g$, $\pm 4g$ și $\pm 8g$, iar aceștia sunt expuși către user-space prin mecanismele subsistemului IIO.

În repaus, un accelerometru nu măsoară doar accelerații produse de mișcare, ci și accelerația gravitațională. Din acest motiv, dacă senzorul este nemișcat, modulul vectorului accelerației trebuie să fie apropiat de valoarea $1g$, iar distribuția acestei valori pe axele X, Y și Z depinde de orientarea fizică a senzorului. Acest comportament este util pentru verificarea funcționării de bază a sistemului, deoarece permite validarea valorilor citite chiar și fără un echipament extern de testare.

În contextul acestei lucrări, accelerometrul nu este tratat doar ca un senzor izolat, ci ca un dispozitiv integrat într-un sistem Linux embedded. Din acest motiv, accentul nu cade doar pe principiul fizic de funcționare, ci și pe modul în care valorile furnizate de senzor sunt accesate prin magistrala I2C, interpretate de un driver kernel și expuse aplicațiilor din spațiul utilizatorului printr-o interfață standardizată.

1.2 Interfața I²C

I2C, prescurtare de la Inter-Integrated Circuit, este o magistrală serială utilizată frecvent pentru comunicația între circuite integrate aflate pe aceeași placă electronică sau în același sistem embedded. Această interfață este potrivită pentru conectarea senzorilor, memoriilor EEPROM, convertoarelor analog-digitale, circuitelor de management al alimentării și altor periferice cu rată de transfer moderată. În cazul lucrării de față, interfața I2C este utilizată pentru comunicația dintre Raspberry Pi 4 Model B și accelerometrul MMA8452Q[4, 5].

Magistrala I2C utilizează două linii principale: SDA, pentru transferul datelor, și SCL, pentru semnalul de ceas. Ambele linii sunt de tip open-drain/open-collector și necesită rezistențe de pull-up pentru a reveni în starea logică de nivel înalt. Comunicația este organizată după un model controller-target, numit tradițional master-slave. Controllerul inițiază transferul, generează semnalul de ceas și adresează dispozitivul cu care dorește să comunice. Dispozitivul adresat răspunde la operațiile de citire sau scriere [6].

Fiecare dispozitiv conectat la magistrala I2C are o adresă, iar controllerul selectează dispozitivul dorit prin transmiterea acestei adrese la începutul tranzacției. În cazul unui senzor digital, comunicația se realizează de obicei prin citirea și scrierea unor registre interne. Pentru citirea unei valori, controllerul transmite mai întâi adresa registrului, apoi inițiază o operație de citire prin care primește unul sau mai mulți octeți de la dispozitiv. Pentru configurare, controllerul scrie valori în registrele de control ale senzorului.

În Linux, dispozitivele I2C sunt reprezentate prin structuri de tip `i2c_client`, iar magistralele I2C sunt reprezentate prin adaptoare. Driverul unui dispozitiv I2C nu controlează direct semnalele electrice SDA și SCL, ci folosește funcțiile oferite de subsistemul I2C al kernelului. Acest lucru permite separarea între driverul controlerului I2C, specific platformei hardware, și driverul dispozitivului conectat la magistrală. În implementarea realizată, driverul pentru MMA8452Q folosește funcții precum `i2c_smbus_read_byte_data`, `i2c_smbus_write_byte_data` și `i2c_smbus_read_i2c_block_data` pentru accesarea registrelor senzorului.

Pentru verificarea comunicației cu senzorul, o etapă importantă este citirea registrului de identificare `WHO_AM_I`. Dacă valoarea citită corespunde valorii așteptate, driverul poate considera că dispozitivul este prezent și funcțional pe magistrala I2C. În cazul implementării din această lucrare, valoarea așteptată este `0x2A`, iar o valoare diferită determină returnarea erorii `-ENODEV`,

indicând faptul că dispozitivul detectat nu este cel așteptat.

Interfața I2C este potrivită pentru acest proiect deoarece MMA8452Q este un senzor digital cu registre interne, iar cantitatea de date transferată este redusă. Pentru fiecare axă, valoarea brută este reprezentată prin doi octeți, corespunzători părților MSB și LSB. Pentru citirea simultană a celor trei axe, driverul poate citi registre consecutive, obținând într-o singură operație valorile pentru X, Y și Z, corespunzători părților MSB și LSB ale valorii brute. Driverul citește aceste valori din registre consecutive și le convertește într-o valoare semnată pe 12 biți, care este apoi expusă către subsistemul IIO.

1.3 Sistemul Linux și modelul driverelor de dispozitiv

Linux este un sistem de operare utilizat frecvent în aplicații embedded datorită flexibilității, suportului extins pentru arhitecturi hardware diferite și existenței unui număr mare de subsisteme dedicate perifericelor. Într-un sistem Linux, accesul direct la hardware este realizat în mod obișnuit de cod care rulează în spațiul kernelului. Aplicațiile obișnuite rulează în user-space și nu accesează direct registrele hardware ale dispozitivelor, ci folosesc interfețele oferite de kernel.

Un driver de dispozitiv are rolul de a face legătura între hardware și restul sistemului de operare. Driverul cunoaște detaliile specifice dispozitivului, cum ar fi registrele, secvența de inițializare și formatul datelor, iar kernelul oferă mecanisme standard prin care aceste funcționalități sunt puse la dispoziția aplicațiilor. Această separare este importantă deoarece permite aplicațiilor să utilizeze dispozitive hardware fără să depindă de detaliile interne ale fiecărui circuit.

Modelul driverelor în Linux este organizat în jurul conceptelor de magistrală, dispozitiv și driver. Magistrala reprezintă mediul prin care dispozitivele sunt conectate la sistem, de exemplu I2C, SPI, USB sau PCI. Dispozitivul reprezintă componenta hardware propriu-zisă, iar driverul este codul care știe să controleze acel dispozitiv. Kernelul asociază un driver cu un dispozitiv pe baza unor informații de identificare, cum ar fi numele dispozitivului, identificatori specifici magistralei sau proprietăți din Device Tree[7].

În cazul unui driver I2C, asocierea dintre dispozitiv și driver duce la apelarea funcției `probe`. Această funcție reprezintă punctul principal de inițializare al driverului. În cadrul ei se verifică prezența dispozitivului, se alocă structurile interne necesare, se configurează hardware-ul și se înregistrează interfața prin care dispozitivul va fi accesibil în sistem. În driverul implementat pentru MMA8452Q, funcția `probe` citește registrul `WHO_AM_I`, alocă un obiect IIO cu `devm_iio_device_alloc`, inițializează structura privată a driverului, configurează senzorul și înregistrează dispozitivul în subsistemul IIO.

O caracteristică importantă a codului kernel este gestionarea atentă a resurselor. În implementarea realizată se folosesc funcții de tip `devm_`, cum ar fi `devm_iio_device_alloc` și `devm_iio_device_register`. Aceste funcții sunt utile deoarece resursele alocate sunt asociate cu dispozitivul și sunt eliberate automat atunci când dispozitivul este eliminat sau driverul este descărcat. Astfel, codul devine mai simplu și mai sigur din punct de vedere al gestionării memoriei.

De asemenea, driverul folosește un mutex pentru protejarea accesului la senzor. Acest lucru este necesar deoarece mai multe operații pot accesa aceleași registre sau aceeași stare internă a driverului. De exemplu, citirea unei axe, modificarea domeniului de măsurare sau schimbarea ratei de eșantionare nu trebuie să se suprapună într-un mod care ar putea produce date inconsistente. În structura privată `mma8452_data`, mutexul este folosit împreună cu informațiile despre clientul I2C și configurația curentă a senzorului [8, 4]. Driverul folosește întreruperi și buffer IIO. De aceea, accesul la senzor trebuie sincronizat.

Citirea din sysfs, configurarea senzorului și handlerul de trigger pot accesa aceleași registre I2C. Pentru a evita suprapunerea operațiilor, accesul este protejat cu un mutex.

Prin urmare, driverul Linux nu reprezintă doar o colecție de funcții pentru citirea unor registre, ci o componentă integrată în modelul general al kernelului. El trebuie să respecte regulile subsistemului I2C, să se integreze cu mecanismele de descoperire a dispozitivelor și să expună datele printr-o interfață standardizată. În această lucrare, această interfață standardizată este oferită de subsistemul Industrial I/O[9].

1.4 Subsistemul Industrial I/O (IIO)

Industrial I/O, prescurtat IIO, este un subsistem al kernelului Linux destinat dispozitivelor care furnizează sau consumă date de măsurare, cum ar fi accelerometre, giroscopae, magnetometre, convertoare analog-digitale, senzori de lumină sau senzori de presiune. Scopul IIO este de a oferi un cadru unificat pentru implementarea driverelor de senzori și o interfață standard către aplicațiile din user-space. Documentația kernelului precizează că IIO este orientat către dispozitive care realizează conversii de tip analog-digital sau digital-analog, dar în practică include și numeroși senzori digitali care furnizează date măsurate [9].

Un avantaj important al subsistemului IIO este standardizarea modului în care datele sunt expuse către user-space. În loc ca fiecare driver să creeze o interfață proprie, IIO definește concepte comune precum dispozitiv IIO, canale, attribute, scale, frecvență de eșantionare, buffer și trigger. Astfel, aplicațiile pot accesa senzori diferiți folosind un model asemănător, iar dezvoltarea driverelor devine mai coerentă.

În IIO, un senzor este reprezentat printr-o structură de tip `iio_dev`. Aceasta conține informații despre numele dispozitivului, canalele disponibile, operațiile suportate și modul de funcționare. Canalele descriu mărimile măsurate de senzor. În cazul unui accelerometru triaxial, există trei canale principale, corespunzătoare axelor X, Y și Z. Fiecare canal poate avea attribute proprii, cum ar fi valoarea brută, și attribute comune, cum ar fi factorul de scalare sau frecvența de eșantionare [10].

În driverul implementat, canalele sunt definite prin structuri `iio_chan_spec`, iar fiecare axă este reprezentată ca un canal de tip `IIO_ACCEL`. Pentru fiecare canal este disponibilă citirea valorii brute prin `IIO_CHAN_INFO_RAW`. În plus, driverul expune informații comune pentru tipul de canal, cum ar fi `IIO_CHAN_INFO_SCALE` și `IIO_CHAN_INFO_SAMP_FREQ`. Acestea permit aplicațiilor să afle factorul de conversie al valorilor brute și rata de eșantionare configurată.

Funcția `read_raw` este una dintre funcțiile centrale ale unui driver IIO. Aceasta este apelată atunci când user-space solicită citirea unei valori prin interfața sysfs. În implementarea realizată, `read_raw` tratează trei cazuri principale: citirea valorii brute a unei axe, citirea scalei și citirea frecvenței de eșantionare. Pentru citirea valorii brute, driverul apelează funcția care citește doi octeți din registrele senzorului și îi convertește într-o valoare semnată pe 12 biți. Pentru scale și sampling frequency, driverul returnează valorile corespunzătoare configurației curente.

Pe lângă citire, driverul implementează și operații de scriere prin `write_raw`. Acest lucru permite modificarea unor parametri de funcționare ai senzorului din user-space. În cazul lucrării de față, pot fi configurate domeniul de măsurare și rata de eșantionare. Valorile disponibile sunt expuse prin `read_avail`, astfel încât aplicațiile să poată determina ce opțiuni sunt acceptate de driver. Această abordare respectă modelul IIO, în care driverul nu oferă doar date brute, ci și informații despre modul corect de interpretare și configurare a acestora[11].

IIO permite două moduri principale de acces la date: citire directă și achiziție prin buffer. Citirea directă este potrivită pentru testare, configurare și citiri individuale din user-space, prin fișierele sysfs. Achiziția prin buffer este utilă atunci când este necesară colectarea unui

flux continuu de date, sincronizat cu un trigger hardware sau software[12, 13].

În implementarea realizată, driverul folosește atât modul de citire directă, cât și modul cu buffer declanșat prin trigger. Modul direct permite citirea valorilor brute prin fișiere precum `in_accel_x_raw`, `in_accel_y_raw` și `in_accel_z_raw`. Modul buffer permite citirea continuă a eșantioanelor prin dispozitivul caracter `/dev/iio:deviceX`.

Pentru achiziția prin buffer, driverul definește canale scanabile pentru axele X, Y și Z și un canal suplimentar de timestamp. La apariția unei întreruperi hardware generate de senzor, driverul declanșează triggerul IIO, citește cele trei axe și introduce eșantionul în buffer împreună cu timestampul asociat. Astfel, aplicația user-space poate citi un flux continuu de date fără să interogheze manual senzorul pentru fiecare valoare.

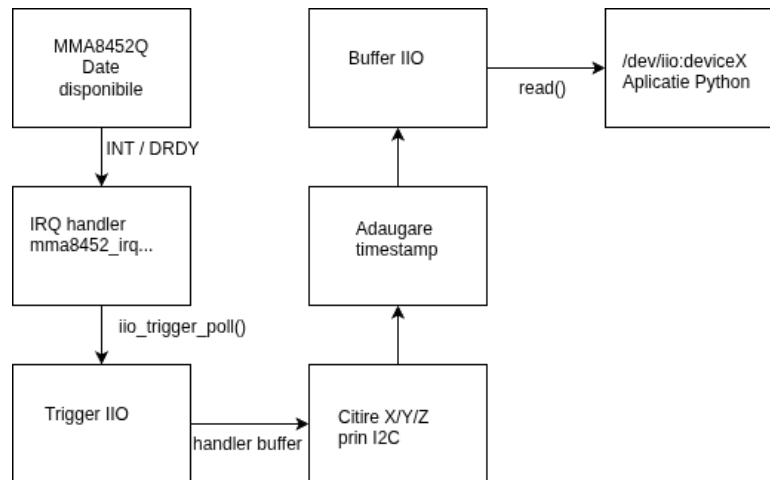


Figura 1.2: Fluxul de achiziție a datelor folosind întrerupere hardware, trigger IIO și buffer

Această abordare este mai apropiată de utilizarea reală a unui senzor într-un sistem embedded, deoarece separă mecanismul de achiziție de aplicația user-space. Driverul gestionează comunicarea cu senzorul, întreruperea și introducerea datelor în buffer, iar aplicația citește datele deja colectate prin interfața standard IIO.

1.5 Device Tree în Linux

Device Tree este un mecanism utilizat în sistemele embedded pentru descrierea hardware-ului către kernel. În multe platforme embedded, dispozitivele conectate la procesor nu pot fi descoperite automat, așa cum se întâmplă în cazul unor magistrale precum PCI sau USB. Din acest motiv, kernelul trebuie să primească o descriere a componentelor existente pe placă, a adreselor acestora, a magistrelor pe care sunt conectate și a proprietăților necesare pentru inițializare. Device Tree oferă această descriere într-o formă structurată[14].

Un fișier Device Tree descrie sistemul hardware sub forma unui arbore de noduri. Fiecare nod poate reprezenta un procesor, o magistrală, un controler sau un dispozitiv periferic. Proprietățile nodurilor oferă informații precum adrese, întreruperi, frecvențe, stări de activare sau șiruri de compatibilitate. Pentru un dispozitiv I2C, nodul este plasat de obicei sub nodul magistralei I2C corespunzătoare și conține proprietăți precum `compatible` și `reg`. Proprietatea `reg` indică adresa dispozitivului pe magistrala I2C, iar `compatible` este folosită pentru asocierea dispozitivului cu driverul potrivit [15].

În Linux, asocierea dintre un dispozitiv descris în Device Tree și un driver se face prin compararea proprietății `compatible` cu lista de compatibilități declarată în driver. În driverul implementat pentru MMA8452Q, această listă este definită prin structura `of_device_id`, care conține intrarea `nxp,mma8452q`. Această intrare este apoi asociată cu driverul I2C prin câmpul

`.of_match_table`. Astfel, atunci când kernelul găsește în Device Tree un dispozitiv cu proprietatea `compatible = "nxp,mma8452q"`, poate lega acel dispozitiv de driverul implementat.

După asocierea dispozitivului cu driverul, kernelul creează o structură `i2c_client` pentru dispozitivul respectiv și apelează funcția `probe` a driverului. Din perspectiva driverului, `i2c_client` conține informațiile necesare pentru comunicația cu senzorul, inclusiv magistrala I2C și adresa dispozitivului. Driverul nu trebuie să caute manual senzorul pe magistrală, ci primește această informație de la kernel, pe baza descrierii din Device Tree.

În cadrul acestei lucrări, Device Tree are un rol important deoarece permite integrarea accelerometrului MMA8452Q în sistemul Armbian rulat pe Raspberry Pi 4 Model B. Prin descrierea senzorului în Device Tree, sistemul poate încărca și asocia automat driverul la pornire, fără ca aplicațiile user-space să fie responsabile de inițializarea directă a hardware-ului. Această abordare este specifică sistemelor Linux embedded și contribuie la separarea clară dintre descrierea platformei hardware, codul driverului și aplicațiile de testare.

În varianta utilizată pentru testarea bufferului IIO, nodul Device Tree al senzorului include și informații despre întreruperea hardware. Senzorul este conectat la un pin GPIO al plăcii Raspberry Pi, iar acest pin este declarat prin proprietățile `interrupt-parent` și `interrupts`. Astfel, kernelul poate asocia linia de întrerupere fizică cu driverul senzorului.

Această configurare permite driverului să primească notificări hardware atunci când senzorul are date noi disponibile. În driver, întreruperea este folosită pentru declanșarea triggerului IIO, iar triggerul determină citirea datelor și introducerea lor în buffer. Astfel, Device Tree nu descrie doar existența senzorului pe magistrala I2C, ci și conexiunea hardware folosită pentru semnalizarea evenimentelor.

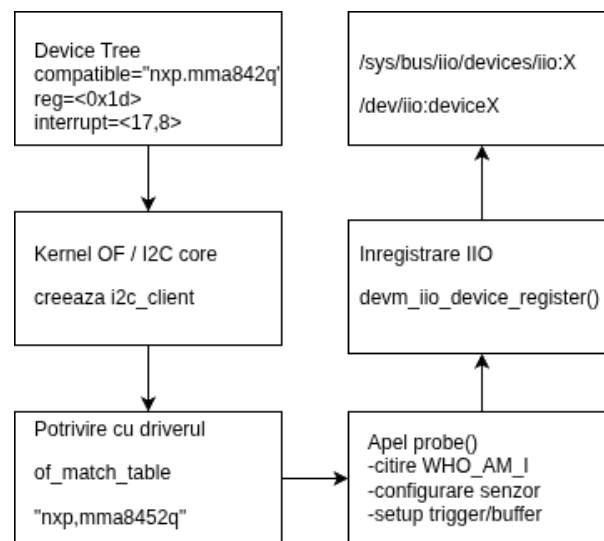


Figura 1.3: Fluxul de asociere a dispozitivului descris în Device Tree cu driverul I2C și înregistrarea acestuia în subsistemul IIO

Device Tree nu înlocuiește driverul, ci îl completează. Driverul conține logica de control a senzorului: citirea registrului `WHO_AM_I`, configurarea domeniului de măsurare, setarea ratei de eșantionare, tratarea întreruperii și citirea valorilor brute. Device Tree descrie faptul că senzorul există în sistem, pe ce magistrală este conectat, ce adresă are, ce driver este compatibil cu el și ce linie de întrerupere folosește. Împreună, aceste două componente permit integrarea corectă a senzorului în kernel și expunerea lui către user-space prin IIO.

Capitolul 2

Platforma hardware și software

Acest capitol prezintă platforma hardware și software utilizată pentru dezvoltarea și testarea driverului Linux Industrial I/O pentru accelerometrul MMA8452Q. Scopul capitolului este de a descrie mediul experimental folosit, modul de conectare al senzorului, verificarea comunicației I2C și declararea dispozitivului în sistem prin Device Tree.

Platforma aleasă trebuie să permită rularea unui sistem Linux complet, accesul la magistrala I2C, utilizarea pinilor GPIO pentru întreruperi și compilarea unui modul kernel. Din acest motiv, în lucrare a fost utilizată o placă Raspberry Pi 4 Model B, împreună cu un modul accelerometru MMA8452Q conectat prin I2C. În varianta finală a implementării, senzorul nu este folosit doar pentru citiri directe prin I2C, ci și pentru achiziția de date prin întreruperi hardware, trigger IIO și buffer IIO.

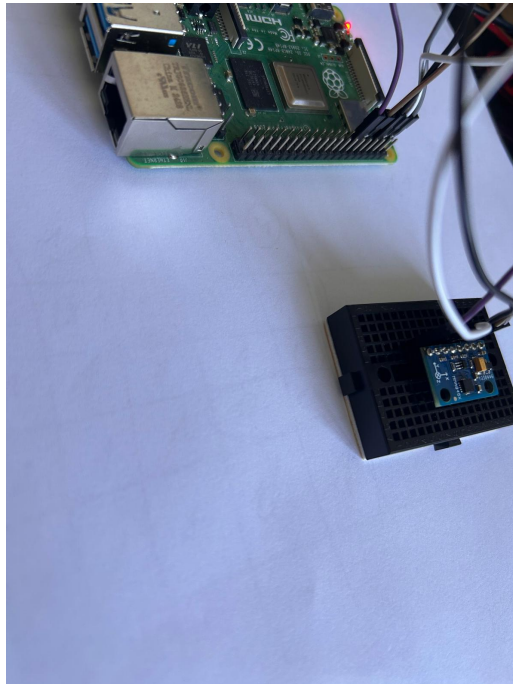


Figura 2.1: Platforma experimentală utilizată: Raspberry Pi 4 Model B și modulul MMA8452Q conectat prin I2C

2.1 Platforma hardware utilizată

Platforma hardware utilizată în această lucrare este formată dintr-o placă Raspberry Pi 4 Model B și un modul cu accelerometrul MMA8452Q. Alegerea plăcii Raspberry Pi 4 Model B a fost făcută deoarece aceasta permite rularea unui sistem Linux complet, oferă acces direct la interfețe hardware prin conectorul GPIO și permite dezvoltarea și testarea modulelor kernel direct pe platforma țintă.

Pentru această lucrare a fost important ca platforma să ofere o magistrală I2C accesibilă din Linux, deoarece accelerometrul MMA8452Q comunică prin interfața I2C. Pe Raspberry Pi 4 Model B, magistrala I2C utilizată pentru senzorii externi este expusă pe pinii GPIO2 și GPIO3, corespunzători semnalelor SDA și SCL. Această magistrală apare în sistemul Linux sub forma dispozitivului `/dev/i2c-1`.

Accelerometrul MMA8452Q este un senzor triaxial digital produs de NXP Semiconductors. Acesta permite măsurarea accelerației pe axele X, Y și Z, are ieșire digitală și comunică prin interfața I2C. Senzorul oferă domenii de măsurare configurabile de $\pm 2g$, $\pm 4g$ și $\pm 8g$, iar datele măsurate sunt disponibile în registre interne care pot fi citite de driver prin magistrala I2C [2].

Pe lângă liniile I2C, modulul MMA8452Q pune la dispoziție și pini de întrerupere. În această lucrare a fost utilizat un pin de întrerupere al senzorului pentru semnalizarea disponibilității unui nou eșantion de date. Acest semnal este folosit ulterior de driver pentru declanșarea triggerului IIO și pentru introducerea datelor în bufferul IIO.

Conectarea senzorului la Raspberry Pi s-a realizat prin semnalele de alimentare, masă, comunicație I2C și întrerupere hardware. Alimentarea modulului s-a făcut la 3,3 V, deoarece pinii GPIO ai Raspberry Pi folosesc niveluri logice de 3,3 V. Conectarea unui semnal de 5 V direct la pinii GPIO nu este sigură pentru Raspberry Pi și poate deteriora placa.

Conexiunile utilizate au fost următoarele:

Pin modul MMA8452Q	Pin Raspberry Pi 4 Model B
VCC	3.3 V
GND	GND
SDA	GPIO2 / pin 3
SCL	GPIO3 / pin 5
INT1	GPIO17 / pin 11

Tabela 2.1: Conectarea accelerometrului MMA8452Q la Raspberry Pi 4 Model B

Această conectare permite procesorului de pe Raspberry Pi să comunice cu senzorul prin controlerul I2C integrat și să primească semnalul de întrerupere atunci când senzorul indică disponibilitatea unor date noi. Verificarea conexiunii nu s-a bazat doar pe schema de conectare, ci a fost realizată practic prin scanarea magistralei I2C și prin citirea registrului de identificare al senzorului, așa cum este prezentat în secțiunea următoare.

2.2 Mediul software utilizat

Platforma software utilizată în cadrul proiectului a fost bazată pe sistemul de operare Armbian, rulat pe Raspberry Pi 4 Model B. Distribuția utilizată a fost Armbian 26.2.1 noble, bazată pe Ubuntu 24.04 LTS Noble Numbat. Această distribuție a fost aleasă deoarece oferă suport pentru plăci ARM, permite acces la interfețele hardware ale plăcii și include infrastructura necesară pentru dezvoltarea de module kernel[1, 16].

Versiunea sistemului de operare a fost verificată cu ajutorul comenzii:

```
cat /etc/os-release
```

Rezultatul relevant obținut a fost:

```
PRETTY_NAME="Armbian 26.2.1 noble"
NAME="Ubuntu"
VERSION_ID="24.04"
VERSION="24.04 LTS (Noble Numbat)"
VERSION_CODENAME=noble
ARMBIAN_PRETTY_NAME="Armbian 26.2.1 noble"
```

Această verificare a fost realizată pentru a documenta exact distribuția Linux pe care a fost dezvoltat și testat driverul. În cazul driverelor kernel, versiunea sistemului și versiunea kernelului sunt importante, deoarece interfețele interne ale kernelului pot diferi între versiuni.

Versiunea kernelului Linux a fost verificată prin comanda:

```
uname -a
```

Rezultatul obținut a fost:

```
Linux 6.18.10-current-bcm2711 #12 SMP PREEMPT
aarch64 GNU/Linux
```

Prin această comandă s-a demonstrat că sistemul rulează un kernel Linux pentru arhitectura `aarch64`, specifică platformei ARM pe 64 de biți. Driverul a fost dezvoltat și testat pe kernelul `6.18.10-current-bcm2711`, folosind antetele kernelului corespunzătoare acestei versiuni.

Mediul software utilizat a inclus următoarele componente:

- subsistemul I2C al kernelului Linux, utilizat pentru comunicația cu senzorul;
- subsistemul Industrial I/O, utilizat pentru expunerea valorilor măsurate către spațiul utilizatorului;
- mecanismul de trigger și buffer din IIO, utilizat pentru achiziția de eșantioane la apariția întreruperilor;
- suportul pentru întreruperi GPIO, utilizat pentru conectarea semnalului de întrerupere al senzorului la kernel;
- suportul Device Tree, utilizat pentru descrierea senzorului conectat pe magistrala I2C;
- utilitarele `i2c-tools`, utilizate pentru testarea comunicației I2C din user-space;
- compilatorul și antetele kernelului, utilizate pentru construirea modulului kernel.

Pentru verificarea comunicației I2C au fost folosite utilitarele `i2cdetect` și `i2cget`. Acestea au fost utilizate înainte de testarea driverului pentru a demonstra că senzorul este vizibil pe magistrala I2C și că răspunde corect la citirea registrului de identificare. Această etapă este importantă deoarece separă problemele hardware de problemele software. Dacă senzorul nu este detectat cu aceste utilitare, problema nu este în driver, ci în alimentare, conexiuni, activarea magistralei sau adresa I2C folosită.

2.3 Conectarea senzorului și verificarea comunicației I2C

După conectarea fizică a senzorului, primul pas a fost verificarea magistralelor I2C disponibile în sistem. Pentru aceasta s-a folosit comanda:

```
ls /dev/i2c-*
```

Rezultatul obținut a fost:

```
/dev/i2c-1 /dev/i2c-20 /dev/i2c-21
```

Acest rezultat demonstrează că sistemul Linux a creat dispozitivele corespunzătoare magistrelor I2C disponibile. Pentru senzorul conectat pe pinii GPIO ai plăcii Raspberry Pi a fost utilizată magistrala `/dev/i2c-1`.

Pentru identificarea controlerelor I2C disponibile s-a folosit și comanda:

```
i2cdetect -l
```

Rezultatul a fost:

i2c-1	unknown	bcm2835 (i2c@7e804000)	N/A
i2c-20	unknown	fef04500.i2c	N/A
i2c-21	unknown	fef09500.i2c	N/A

Această comandă a fost utilizată pentru a confirma existența magistralei I2C principale, controlată de driverul `bcm2835`, și pentru a identifica magistrala care trebuie scanată pentru senzorul extern.

Scanarea magistralei I2C 1 s-a realizat cu următoarea comandă:

```
sudo i2cdetect -y 1
```

Rezultatul obținut a fost:

```

    0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:                -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- 1c -- -- --
20: -- -- -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- -- -- --
40: -- -- -- -- -- -- -- -- -- -- -- -- -- --
50: -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- --
```

Prin această scanare s-a demonstrat că pe magistrala `/dev/i2c-1` există un dispozitiv care răspunde la adresa `0x1c`. Această adresă este adresa I2C utilizată de modulul MMA8452Q în configurația hardware folosită în lucrare.

Apariția unei adrese în rezultatul comenzii `i2cdetect` demonstrează doar că există un dispozitiv pe magistrală, nu și faptul că acel dispozitiv este accelerometrul MMA8452Q. Din acest motiv, verificarea a fost completată prin citirea registrului de identificare al senzorului.

Pentru MMA8452Q, registrul `WHO_AM_I` se află la adresa internă `0x0D`, iar valoarea așteptată este `0x2A` [2]. Citirea acestui registru s-a realizat cu următoarea comandă:

```
sudo i2cget -y 1 0x1c 0x0d
```

Rezultatul obținut a fost:

0x2a

Acest rezultat demonstrează că dispozitivul detectat la adresa 0x1c este accelerometrul MMA8452Q. Verificarea este importantă deoarece aceeași valoare este folosită și în funcția `probe` a driverului. În momentul în care driverul este asociat cu dispozitivul, acesta citește registrul `WHO_AM_I` și compară valoarea citită cu valoarea așteptată, 0x2A. Dacă valoarea nu corespunde, driverul returnează eroarea `-ENODEV`, indicând faptul că dispozitivul nu este compatibil.

Prin urmare, comenzile `i2cdetect` și `i2cget` au fost folosite pentru a demonstra două lucruri diferite. Prima comandă a demonstrat că senzorul este vizibil pe magistrala I2C la adresa 0x1c. A doua comandă a demonstrat că dispozitivul de la acea adresă este într-adevăr MMA8452Q, deoarece registrul său de identificare a returnat valoarea corectă.

Această etapă a fost realizată înainte de testarea driverului pentru a confirma că partea hardware și comunicația I2C funcționează corect. Astfel, eventualele erori apărute ulterior la încărcarea driverului pot fi analizate la nivel software, fără a confunda problemele de implementare cu probleme de conectare fizică.

2.4 Declararea senzorului și a întreruperii prin Device Tree

Pentru ca driverul kernel să fie apelat automat, nu este suficient ca senzorul să fie vizibil electric pe magistrala I2C. Kernelul trebuie să cunoască existența dispozitivului, adresa acestuia pe magistrala I2C și, în cazul utilizării întreruperilor hardware, pinul GPIO la care este conectată linia de întrerupere a senzorului. În această lucrare, aceste informații au fost furnizate kernelului printr-un overlay Device Tree.

Overlay-ul Device Tree descrie senzorul MMA8452Q ca dispozitiv conectat pe magistrala `i2c1`, la adresa 0x1c. În plus, overlay-ul descrie și linia de întrerupere utilizată pentru semnalizarea disponibilității datelor, astfel încât driverul să poată primi valoarea corespunzătoare în câmpul `client->irq`.

Overlay-ul Device Tree utilizat a fost compilat în fișierul:

```
/boot/firmware/overlays/mma8452q-overlay.dtbo
```

Activarea overlay-ului s-a realizat prin adăugarea următoarei linii în fișierul:

```
/boot/firmware/config.txt
```

Linia adăugată a fost:

```
dtoverlay=mma8452q-overlay
```

Această modificare a fost făcută pentru ca, la pornirea sistemului, overlay-ul să fie aplicat peste arborele Device Tree principal. În urma aplicării overlay-ului, kernelul primește informația că pe magistrala I2C 1 există un dispozitiv cu adresa 0x1c, cu proprietatea `compatible = "nxp,mma8452q"` și cu o linie de întrerupere conectată la controlerul GPIO.

În forma sursă, overlay-ul utilizat a avut următoarea structură:

```

/dts-v1/;
/plugin/;

#include <dt-bindings/interrupt-controller/irq.h>

/ {
    compatible = "brcm,bcm2711";

    fragment@0 {
        target = <&i2c1>;

        __overlay__ {
            #address-cells = <1>;
            #size-cells = <0>;
            status = "okay";

            mma8452q@1c {
                compatible = "nxp,mma8452q";
                reg = <0x1c>;

                interrupt-parent = <&gpio>;
                interrupts = <17 8>;

                status = "okay";
            };
        };
    };
};

```

Nodul `mma8452q@1c` descrie senzorul conectat pe magistrala I2C. Proprietățile principale ale nodului sunt:

```

reg = <0x1c>;
compatible = "nxp,mma8452q";

```

Proprietatea `reg` indică adresa dispozitivului pe magistrală, aceeași adresă observată anterior cu `i2cdetect`. Proprietatea `compatible` este folosită de kernel pentru asocierea dispozitivului cu driverul corespunzător.

Proprietatea `interrupt-parent = <&gpio>` indică faptul că întreruperea este gestionată de controlerul GPIO al plăcii Raspberry Pi, iar proprietatea `interrupts` precizează pinul GPIO utilizat și tipul de declanșare al acesteia. În acest caz, pinul GPIO17 este utilizat pentru semnalul de întrerupere al senzorului. Tipul de întrerupere a fost configurat ca `IRQ_TYPE_LEVEL_LOW`, deoarece linia de întrerupere rămâne activă până la citirea registrelor care confirmă evenimentul de tip `data-ready`.

Această proprietate este legată direct de codul driverului, unde este definită tabela de compatibilitate:

```

static const struct of_device_id mma8452_of_match[] = {
    { .compatible = "nxp,mma8452q" },
    { }
};

MODULE_DEVICE_TABLE(of, mma8452_of_match);

```

Prin urmare, overlay-ul Device Tree demonstrează legătura dintre descrierea hardware a senzorului și driverul implementat. La pornirea sistemului, kernelul aplică overlay-ul, creează dispozitivul I2C corespunzător nodului `mma8452q@1c`, compară proprietatea `compatible` cu tabela `of_device_id` din driver și, dacă există potrivire, apelează funcția `probe` a driverului.

Driverul folosește și informația despre întrerupere furnizată prin Device Tree. În funcția `probe`, se verifică dacă există o linie de întrerupere validă, folosind câmpul `client->irq`.

Dacă această valoare este validă, driverul înregistrează un handler de întrerupere folosind funcția `devm_request_threaded_irq`. Astfel, descrierea Device Tree nu este utilizată doar pentru identificarea dispozitivului, ci și pentru configurarea mecanismului de achiziție bazat pe întreruperi.

Pentru verificarea conținutului overlay-ului compilat, fișierul `.dtbo` poate fi decompilat cu utilitarul `dtc`:

```
dtc -I dtb -O dts -o /tmp/mma8452q-overlay.dts \  
/boot/firmware/overlays/mma8452q-overlay.dtbo  
  
cat /tmp/mma8452q-overlay.dts
```

În fișierul decompilat, referința simbolică `<&i2c1>` poate apărea sub forma unei valori numerice temporare, de exemplu `target = <0xffffffff>`. Aceasta nu reprezintă o eroare, deoarece referința este rezolvată prin secțiunea `__fixups__` a overlay-ului. În forma sursă, ținta reală a fragmentului este magistrala `i2c1`.

Această etapă a fost necesară pentru a demonstra că senzorul nu este doar conectat fizic și vizibil pe I2C, ci este și descris corect pentru mecanismul de detecție al kernelului Linux. Astfel, driverul nu trebuie apelat manual pentru o adresă arbitrară, ci este asociat automat cu dispozitivul declarat în Device Tree. În plus, prin includerea proprietăților de întrerupere, driverul primește informațiile necesare pentru achiziția de date prin mecanismul de întreruperi, trigger și buffer IIO.

Capitolul 3

Proiectarea și implementarea driverului IIO

Acest capitol prezintă proiectarea și implementarea driverului Linux pentru accelerometrul MMA8452Q. Scopul implementării a fost integrarea senzorului într-un sistem Linux embedded prin magistrala I2C și expunerea valorilor măsurate prin subsistemul Industrial I/O. Driverul realizat identifică dispozitivul pe magistrala I2C, configurează parametrii principali ai senzorului și permite citirea valorilor brute ale accelerației pe axele X, Y și Z.

Implementarea a fost realizată sub forma unui modul kernel scris în limbajul C. Driverul folosește infrastructura I2C a kernelului pentru comunicarea cu senzorul și infrastructura IIO pentru expunerea datelor către spațiul utilizatorului. Astfel, aplicațiile user-space nu accesează direct registrele senzorului, ci folosesc interfața standard oferită de Linux în directorul `/sys/bus/iio/devices/`. [4, 10]

Pe lângă citirea directă a valorilor prin fișierele `sysfs`, driverul implementează și mecanismul de achiziție bazat pe întreruperi hardware, trigger IIO și buffer IIO. În această configurație, senzorul generează o întrerupere atunci când sunt disponibile date noi, iar driverul citește valorile celor trei axe și le introduce într-un buffer împreună cu un marcaj temporal[17].

3.1 Analiza registrelor relevante ale senzorului MMA8452Q

Pentru implementarea driverului nu a fost necesară utilizarea tuturor registrelor disponibile în MMA8452Q. Au fost selectate doar registrele necesare pentru identificarea senzorului, configurarea domeniului de măsurare, configurarea frecvenței de eșantionare, activarea întreruperii de tip data-ready și citirea datelor de accelerație. Această alegere limitează complexitatea driverului și permite validarea funcțiilor esențiale ale senzorului.

Registrele utilizate în implementare sunt prezentate în tabelul 3.1.

Registrul de identificare al senzorului a fost folosit pentru verificarea dispozitivului. Pentru MMA8452Q, valoarea așteptată este `0x2A`. Această verificare previne atașarea driverului la un dispozitiv greșit de pe magistrala I2C.

Registrul `XYZ_DATA_CFG` este folosit pentru alegerea domeniului de măsurare. În implementare au fost tratate cele trei domenii principale ale senzorului: $\pm 2g$, $\pm 4g$ și $\pm 8g$. Domeniul ales influențează factorul de scalare folosit pentru interpretarea valorilor brute.

Registrul `CTRL_REG1` are două roluri în driver. Primul rol este activarea sau trecerea senzorului în standby prin bitul `ACTIVE`. Al doilea rol este configurarea frecvenței de eșantionare prin câmpul `DR[2:0]`. În implementare, modificarea domeniului de măsurare sau a frecvenței de eșantionare se face cu senzorul în standby, apoi senzorul este reactivat. Această ordine respectă modul normal de configurare al senzorului [2].

Registru	Adresă	Rol în driver
STATUS	0x00	registru de stare al senzorului
OUT_X_MSB	0x01	începutul zonei de date pentru axele X, Y, Z
INT_SOURCE	0x0C	identificarea sursei întreruperii
WHO_AM_I	0x0D	identificarea dispozitivului
XYZ_DATA_CFG	0x0E	configurarea domeniului de măsurare
CTRL_REG1	0x2A	activarea senzorului și configurarea ODR
CTRL_REG4	0x2D	activarea surselor de întrerupere
CTRL_REG5	0x2E	rutarea întreruperilor către pinul INT

Tabela 3.1: Registre MMA8452Q utilizate în driver

Pentru funcționarea pe bază de întreruperi au fost utilizate registrele CTRL_REG4, CTRL_REG5 și INT_SOURCE. Registrul CTRL_REG4 activează sursa de întrerupere de tip data-ready, iar registrul CTRL_REG5 stabilește rutarea acestei întreruperi către pinul de întrerupere al senzorului. Registrul INT_SOURCE este citit în handlerul de întrerupere pentru a verifica dacă întreruperea a fost produsă de apariția unor date noi.

3.2 Structura driverului Linux implementat

Driverul este organizat ca un driver I2C și este înregistrat în kernel prin macro-ul:

```
module_i2c_driver
```

Această structură permite kernelului să asocieze automat driverul cu dispozitivul descris în Device Tree. Asocierea se face atunci când proprietatea `compatible` a dispozitivului corespunde cu lista acceptată de driver.

Structura principală folosită pentru datele interne ale driverului este:

```
struct mma8452_data {
    struct i2c_client *client;
    struct mutex lock;
    u8 fs_bits;
    u8 odr_bits;
    struct iio_trigger *trig;
};
```

Câmpul `client` reprezintă dispozitivul I2C creat de kernel pentru senzor. Prin acest obiect sunt efectuate citirile și scrierile pe magistrala I2C. Câmpul `lock` este folosit pentru protejarea accesului la senzor atunci când se citesc sau se modifică date. Câmpurile `fs_bits` și `odr_bits` memorează configurația curentă a senzorului, respectiv domeniul de măsurare și frecvența de eșantionare. Câmpul `trig` reține triggerul IIO folosit pentru achiziția bazată pe întreruperi.

Pentru datele introduse în bufferul IIO a fost definită structura:

```
struct mma8452_scan {
    s16 channels[3];
    s64 timestamp __aligned(8);
};
```

Vectorul `channels` conține valorile brute pentru axele X, Y și Z. Câmpul `timestamp` este folosit pentru marcajul temporal asociat fiecărui set de date. Alinierea la 8 octeți este necesară deoarece timestampul este o valoare pe 64 de biți și trebuie plasat corect în structura de scanare folosită de IIO.

Driverul conține mai multe funcții cu roluri separate:

- `mma8452_probe()` – identifică senzorul, alocă dispozitivul IIO, configurează triggerul și înregistrează dispozitivul în kernel;
- Funcția `mma8452_chip_init()` configurează valorile inițiale ale senzorului și activează întreruperea data-ready;
- `mma8452_set_active()` – activează sau dezactivează senzorul;
- `mma8452_set_range()` – modifică domeniul de măsurare;
- `mma8452_set_odr()` – modifică frecvența de eșantionare;
- `mma8452_read_axis()` – citește valoarea brută pentru o singură axă;
- `mma8452_read_xyz()` – citește valorile brute pentru toate cele trei axe;
- `mma8452_read_raw()` – expune citirea datelor către subsistemul IIO;
- `mma8452_write_raw()` – permite modificarea parametrilor configurabili din IIO;
- `mma8452_irq_thread()` – tratează întreruperea hardware generată de senzor;
- `mma8452_trigger_handler()` – citește datele și le introduce în bufferul IIO.

Această împărțire face driverul mai ușor de urmărit: funcțiile de nivel jos comunică direct cu registrele senzorului, funcțiile IIO transformă aceste operații în interfețe standard vizibile din user-space, iar funcțiile asociate întreruperilor și bufferului realizează achiziția periodică a datelor.

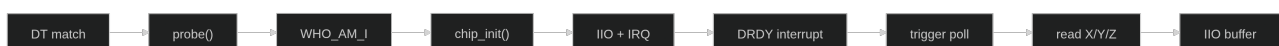


Figura 3.1: Schema logică de funcționare a driverului MMA8452Q

3.3 Inițializarea și identificarea dispozitivului

Inițializarea driverului începe în funcția `probe`. Această funcție este apelată de kernel atunci când există un dispozitiv I2C compatibil cu driverul. În cazul acestei lucrări, dispozitivul este descris în Device Tree prin proprietatea:

```
compatible = "nxp,mma8452q";
```

Driverul conține aceeași valoare în tabela `of_device_id`:

```
static const struct of_device_id mma8452_of_match[] = {
    { .compatible = "nxp,mma8452q" },
    { }
};
```

Această asociere permite kernelului să lege dispozitivul definit în Device Tree de driverul implementat. Device Tree este folosit în Linux pentru descrierea componentelor hardware care nu pot fi descoperite automat de sistem[14].

Primul pas realizat în `probe` este citirea registrului `WHO_AM_I`:

```
ret = i2c_smbus_read_byte_data(client, MMA8452_WHO_AM_I);
```

Dacă citirea eșuează, driverul oprește inițializarea și returnează codul de eroare primit. Dacă citirea reușește, valoarea este comparată cu `0x2A`. În cazul în care valoarea citită este diferită, driverul returnează `-ENODEV`. Această verificare demonstrează că driverul nu se bazează doar pe existența unei adrese I2C, ci verifică explicit identitatea senzorului.

După identificarea corectă a dispozitivului, driverul alocă structura IIO:

```
indio_dev = devm_iio_device_alloc(&client->dev, sizeof(*data));
```

Funcția alocă obiectul IIO și zona privată de memorie a driverului. În această zonă este stocată structura `mma8452_data`, care conține starea internă a senzorului.

Accesul la zona privată se face prin:

```
data = iio_priv(indio_dev);
```

Prin această asociere, obiectul IIO expus de kernel și datele interne ale driverului rămân legate pe toată durata de funcționare a dispozitivului.

În continuare, driverul completează câmpurile principale ale obiectului IIO:

```
indio_dev->name = "mma8452q";
indio_dev->info = &mma8452_iio_info;
indio_dev->modes = INDIO_DIRECT_MODE | INDIO_BUFFER_TRIGGERED;
indio_dev->channels = mma8452_channels;
indio_dev->num_channels = ARRAY_SIZE(mma8452_channels);
indio_dev->available_scan_masks = mma8452_scan_masks;
```

Prin aceste câmpuri, driverul declară numele dispozitivului, operațiile IIO disponibile, modurile de funcționare, canalele de măsurare și combinația de canale permisă pentru buffer.

3.4 Configurarea senzorului

După identificarea dispozitivului, driverul configurează senzorul prin funcția `mma8452_chip_init`. În această etapă, senzorul este trecut inițial în standby.

Apoi se configurează domeniul de măsurare, întreruperea de tip `data-ready` și frecvența de eșantionare.

Configurarea inițială folosită în driver este:

- domeniu de măsurare: $\pm 2g$;
- frecvență de eșantionare: 100 Hz;
- întrerupere activată: `data-ready`;
- mod de funcționare: activ.

Secvența principală este:


```

i2c_smbus_write_byte_data(data->client, MMA8452_CTRL_REG1, 0x00);
i2c_smbus_write_byte_data(data->client, MMA8452_XYZ_DATA_CFG, 0x00);
i2c_smbus_write_byte_data(data->client, MMA8452_CTRL_REG4,
                           MMA8452_INT_DRDY);
i2c_smbus_write_byte_data(data->client, MMA8452_CTRL_REG5,
                           MMA8452_INT_DRDY);
i2c_smbus_write_byte_data(data->client, MMA8452_CTRL_REG1,
                           (3 << MMA8452_CTRL_REG1_DR_SHIFT) | MMA8452_CTRL_REG1_ACTIVE);

```

Prima scriere dezactivează senzorul, a doua scriere setează domeniul de măsurare la $\pm 2g$, următoarele două scrieri activează și rutează întreruperea data-ready, iar ultima scriere setează frecvența de eșantionare la 100 Hz și activează senzorul. Valorile sunt memorate și în structura internă:

```

data->fs_bits = 0;
data->odr_bits = 3;

```

Pe lângă inițializarea implicită, driverul permite modificarea domeniului de măsurare și a frecvenței de eșantionare prin funcțiile `mma8452_set_range` și `mma8452_set_odr`. Ambele funcții folosesc aceeași logică: senzorul este trecut în standby, se modifică registrul corespunzător, apoi senzorul este reactivat.

Pentru domeniul de măsurare, driverul acceptă doar valorile 0, 1 și 2, corespunzătoare domeniilor $\pm 2g$, $\pm 4g$ și $\pm 8g$. Dacă se primește o valoare invalidă, funcția returnează `-EINVAL`. Pentru frecvența de eșantionare, driverul acceptă valorile codificate pe trei biți în câmpul `DR[2:0]` din `CTRL_REG1`.

3.5 Citirea datelor și conversia valorilor brute

Datele de accelerație sunt citite din registrele de ieșire ale senzorului. Pentru fiecare axă sunt folosiți doi octeți: MSB și LSB. În implementare, adresa de început este calculată pornind de la registrul `OUT_X_MSB`:

```

u8 reg = MMA8452_OUT_X_MSB + axis * 2;

```

Astfel, pentru axa X se citește de la `OUT_X_MSB`, pentru axa Y de la următoarea pereche de registre, iar pentru axa Z de la perechea corespunzătoare acestei axe. Citirea unei singure axe se face printr-o operație I2C block read:

```

ret = i2c_smbus_read_i2c_block_data(data->client, reg, 2, buf);

```

Pentru citirea simultană a celor trei axe, folosită în handlerul de trigger, driverul citește șase octeți începând de la registrul `OUT_X_MSB`:

```

ret = i2c_smbus_read_i2c_block_data(data->client,
                                     MMA8452_OUT_X_MSB,
                                     sizeof(buf),
                                     buf);

```

După citirea octeților, valorile sunt convertite în numere semnate pe 12 biți. Funcția care realizează conversia este:

```
static s16 mma8452_convert_12bit(u8 msb, u8 lsb)
{
    u16 raw16 = ((u16)msb << 8) | lsb;
    s16 raw12 = (s16)(raw16 >> 4);

    if (raw12 & 0x0800)
        raw12 |= 0xF000;

    return raw12;
}
```

Această funcție combină cei doi octeți într-o valoare pe 16 biți, elimină cei 4 biți neutilizați prin deplasare la dreapta și extinde semnul valorii pe 12 biți. Extinderea semnului este necesară deoarece accelerația poate avea valori pozitive sau negative, în funcție de orientarea senzorului pe fiecare axă.

Driverul returnează către IIO valoarea brută, nu valoarea direct exprimată în unități fizice. Conversia către accelerație se face folosind factorul de scalare expus separat prin atributul `scale`. Pentru cele trei domenii de măsurare, driverul definește următorii factori de scalare:

```
static const int mma8452_scale_nano[] = {
    976562,
    1953125,
    3906250,
};
```

Valorile sunt exprimate în g pe LSB prin interfața IIO, folosind formatul `IIO_VAL_INT_PLUS_NANO`. Acestea corespund domeniilor $\pm 2g$, $\pm 4g$ și $\pm 8g$.

Valoarea fizică se obține prin înmulțirea valorii brute cu factorul de scalare corespunzător.

3.6 Integrarea cu subsistemul IIO

Integrarea cu subsistemul Industrial I/O este realizată prin definirea canalelor IIO, a structurii `iio_info` și prin înregistrarea dispozitivului IIO în kernel. Subsistemul IIO folosește structura `iio_chan_spec` pentru descrierea canalelor unui senzor. În cazul acestui driver, au fost definite trei canale de tip `IIO_ACCEL`, corespunzătoare axelor X, Y și Z [10].

Canalele sunt definite prin macro-ul:

```
#define MMA8452_ACCEL_CHAN(_axis, _addr) \
{ \
    .type = IIO_ACCEL, \
    .modified = 1, \
    .channel2 = IIO_MOD_##_axis, \
    .address = _addr, \
    .info_mask_separate = BIT(IIO_CHAN_INFO_RAW), \
    .info_mask_shared_by_type = \
        BIT(IIO_CHAN_INFO_SCALE) | \
        BIT(IIO_CHAN_INFO_SAMP_FREQ), \
    .info_mask_shared_by_type_available = \
        BIT(IIO_CHAN_INFO_SCALE) | \
        BIT(IIO_CHAN_INFO_SAMP_FREQ), \
}
```

```

        .scan_index = _addr,
        .scan_type = {
            .sign = 's',
            .realbits = 12,
            .storagebits = 16,
            .shift = 0,
            .endianness = IIO_CPU,
        },
    },
}

```

Prin această definiție, fiecare axă are un atribut separat pentru valoarea brută, iar parametrii `scale` și `sampling-frequency` sunt comuni pentru toate canalele de accelerație. În plus, câmpurile `scan_index` și `scan_type` descriu modul în care datele sunt așezate în bufferul IIO.

Vectorul de canale este:

```

static const struct iio_chan_spec mma8452_channels[] = {
    MMA8452_ACCEL_CHAN(X, 0),
    MMA8452_ACCEL_CHAN(Y, 1),
    MMA8452_ACCEL_CHAN(Z, 2),
    IIO_CHAN_SOFT_TIMESTAMP(3),
};

```

Primele trei canale corespund axelor de accelerație, iar ultimul canal reprezintă timestampul software asociat fiecărui set de date introdus în buffer.

Operațiile IIO sunt definite în structura:

```

static const struct iio_info mma8452_iio_info = {
    .read_raw = mma8452_read_raw,
    .write_raw = mma8452_write_raw,
    .write_raw_get_fmt = mma8452_write_raw_get_fmt,
    .read_avail = mma8452_read_avail,
};

```

Funcția `read_raw` este apelată atunci când utilizatorul citește valorile expuse în sysfs. Driverul tratează trei tipuri principale de cereri:

- `IIO_CHAN_INFO_RAW` – citește direct axa cerută din senzor;
- `IIO_CHAN_INFO_SCALE` – returnează factorul de scalare corespunzător domeniului curent;
- `IIO_CHAN_INFO_SAMP_FREQ` – returnează frecvența de eșantionare curentă.

Funcția `write_raw` permite modificarea parametrilor configurabili. Dacă utilizatorul scrie o valoare validă pentru `scale`, driverul schimbă domeniul de măsurare. Dacă utilizatorul scrie o valoare validă pentru `sampling-frequency`, driverul schimbă frecvența de eșantionare. Valorile acceptate sunt expuse prin funcția `read_avail`, ceea ce permite utilizatorului să vadă direct ce configurații sunt suportate.

În final, dispozitivul IIO este înregistrat în funcția `probe`:

```

ret = devm_iio_device_register(&client->dev, indio_dev);

```

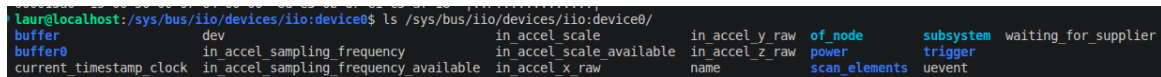
După înregistrare, kernelul creează automat interfața IIO în user-space. Astfel, valorile senzorului pot fi citite din fișiere precum:

```
in_accel_x_raw
in_accel_y_raw
in_accel_z_raw
in_accel_scale
in_accel_sampling_frequency
```

De asemenea, valorile disponibile pentru configurare sunt expuse prin fișierele:

```
in_accel_scale_available
in_accel_sampling_frequency_available
```

Această integrare demonstrează funcționarea lanțului hardware–kernel–user-space: senzorul este detectat prin I2C, identificat prin registrul WHO_AM_I, configurat prin registrele interne și expus către aplicații prin interfața standard Industrial I/O.



```
laure@localhost:/sys/bus/iio/devices/iio:device0$ ls /sys/bus/iio/devices/iio:device0/
buffer      dev          in_accel_scale      in_accel_y_raw      of_node      subsystem  waiting_for_supplier
buffer0     in_accel_sampling_frequency  in_accel_scale_available  in_accel_z_raw      power        trigger
current_timestamp_clock  in_accel_sampling_frequency_available  in_accel_x_raw      name              scan_elements  uevent
```

Figura 3.2: Atributele sysfs expuse de driverul IIO pentru accelerometrul MMA8452Q

3.7 Implementarea întreruperilor hardware

Pentru achiziția automată a datelor, driverul folosește întreruperea hardware generată de senzor atunci când un nou set de date este disponibil. În acest scop, în funcția de inițializare sunt configurate registrele CTRL_REG4 și CTRL_REG5:

```
i2c_smbus_write_byte_data(data->client,
                           MMA8452_CTRL_REG4,
                           MMA8452_INT_DRDY);
```

```
i2c_smbus_write_byte_data(data->client,
                           MMA8452_CTRL_REG5,
                           MMA8452_INT_DRDY);
```

Prin aceste scrieri, sursa de întrerupere data-ready este activată și rutată către pinul de întrerupere al senzorului. În Device Tree, pinul de întrerupere al senzorului este asociat cu un GPIO al platformei, iar kernelul transformă această descriere într-un număr de IRQ disponibil în câmpul `client->irq`.

În funcția `probe`, driverul verifică existența unui IRQ valid:

```
if (client->irq > 0) {
    ret = devm_request_threaded_irq(&client->dev,
                                    client->irq,
                                    NULL,
                                    mma8452_irq_thread,
                                    IRQF_ONESHOT,
                                    "mma8452q",
                                    indio_dev);
}
```

A fost folosită o întrerupere de tip threaded IRQ. Handlerul rapid este NULL, iar tratarea efectivă este realizată în funcția `mma8452_irq_thread`. Această alegere este potrivită deoarece în handler se face acces I2C, iar operațiile I2C pot dormi și nu trebuie executate într-un handler de întrerupere hard IRQ.

Funcția care tratează întreruperea este:

```
static irqreturn_t mma8452_irq_thread(int irq, void *dev_id)
{
    struct iio_dev *indio_dev = dev_id;
    struct mma8452_data *data = iio_priv(indio_dev);
    int source;

    mutex_lock(&data->lock);
    source = i2c_smbus_read_byte_data(data->client, MMA8452_INT_SOURCE);
    mutex_unlock(&data->lock);

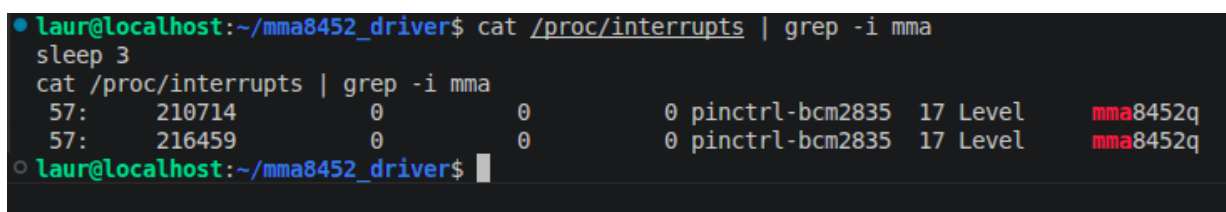
    if (source < 0)
        return IRQ_HANDLED;

    if ((source & MMA8452_INT_DRDY) && data->trig)
        iio_trigger_poll(data->trig);

    return IRQ_HANDLED;
}
```

Handlerul citește registrul `INT_SOURCE` pentru a identifica sursa întreruperii. Dacă bitul `MMA8452_INT_DRDY` este setat, înseamnă că senzorul are date noi disponibile. În acest caz, driverul apelează `iio_trigger_poll`, care pornește mecanismul IIO de achiziție prin trigger.

Această structură separă tratarea întreruperii hardware de citirea efectivă a datelor. Handlerul de IRQ doar confirmă sursa întreruperii și declanșează triggerul IIO, iar citirea valorilor și introducerea lor în buffer se face în handlerul de trigger.



```
• laur@localhost:~/mma8452_driver$ cat /proc/interrupts | grep -i mma
sleep 3
cat /proc/interrupts | grep -i mma
57:      210714      0      0      0 pinctrl-bcm2835 17 Level      mma8452q
57:      216459      0      0      0 pinctrl-bcm2835 17 Level      mma8452q
• laur@localhost:~/mma8452_driver$
```

Figura 3.3: Verificarea întreruperilor generate de accelerometrul în `/proc/interrupts`

3.8 Implementarea triggerului și bufferului IIO

Pentru a permite achiziția continuă a datelor, driverul implementează un trigger IIO și un buffer IIO. Triggerul reprezintă evenimentul care determină citirea unui nou set de date, iar bufferul reprezintă zona prin care aceste date sunt puse la dispoziția aplicațiilor user-space.

Triggerul este alocat și înregistrat în funcția `probe`:

```
data->trig = devm_iio_trigger_alloc(&client->dev,
                                   "%s-trigger",
```

```

        indio_dev->name);

data->trig->ops = &mma8452_trigger_ops;
iio_trigger_set_drvdata(data->trig, indio_dev);

ret = devm_iio_trigger_register(&client->dev, data->trig);

```

Operațiile triggerului sunt definite prin structura:

```

static const struct iio_trigger_ops mma8452_trigger_ops = {
    .validate_device = iio_trigger_validate_own_device,
};

```

Funcția `iio_trigger_validate_own_device` limitează folosirea triggerului la dispozitivul care l-a creat. Această alegere este potrivită pentru implementarea de față, deoarece triggerul este generat de întreruperea senzorului MMA8452Q și este destinat aceluiași dispozitiv IIO.

După înregistrarea triggerului, acesta este asociat cu dispozitivul IIO:

```

indio_dev->trig = iio_trigger_get(data->trig);

```

Bufferul IIO este configurat prin funcția:

```

ret = devm_iio_triggered_buffer_setup(&client->dev,
                                     indio_dev,
                                     iio_pollfunc_store_time,
                                     mma8452_trigger_handler,
                                     NULL);

```

În această configurație, handlerul de trigger este funcția care citește datele senzorului și le introduce în bufferul IIO. Aceasta este apelată atunci când triggerul este activat.

Activarea triggerului are loc după detectarea unei întreruperi de tip `data-ready`. În acel moment, handlerul de IRQ notifică subsistemul IIO prin funcția `iio_trigger_poll[18, 19]`.

Funcția de tratare a triggerului este:

```

static irqreturn_t mma8452_trigger_handler(int irq, void *p)
{
    struct iio_poll_func *pf = p;
    struct iio_dev *indio_dev = pf->indio_dev;
    struct mma8452_data *data = iio_priv(indio_dev);
    struct mma8452_scan scan;
    s16 x, y, z;
    int ret;

    memset(&scan, 0, sizeof(scan));

    mutex_lock(&data->lock);
    ret = mma8452_read_xyz(data, &x, &y, &z);
    mutex_unlock(&data->lock);

    if (ret < 0)
        goto done;
}

```

```

    scan.channels[0] = x;
    scan.channels[1] = y;
    scan.channels[2] = z;

    iio_push_to_buffers_with_timestamp(indio_dev,
                                      &scan,
                                      iio_get_time_ns(indio_dev));

done:
    iio_trigger_notify_done(indio_dev->trig);

    return IRQ_HANDLED;
}

```

În această funcție, driverul citește cele trei axe printr-o singură citire I2C de șase octeți. Datele sunt apoi copiate în structura folosită pentru eșantionul IIO.

Introducerea datelor în buffer se face prin funcția:

`iio_push_to_buffers_with_timestamp`

Timestampul permite corelarea fiecărui set de valori cu momentul achiziției.

La final, funcția apelează `iio_trigger_notify_done`, pentru a informa subsistemul IIO că tratarea triggerului s-a terminat. Acest pas este necesar pentru funcționarea corectă a mecanismului de trigger.

Activarea bufferului se face din user-space prin fișierele create de IIO în directorul dispozitivului. În mod tipic, se activează canalele dorite, se asociază triggerul curent și apoi se pornește bufferul:

```

echo 1 > scan_elements/in_accel_x_en
echo 1 > scan_elements/in_accel_y_en
echo 1 > scan_elements/in_accel_z_en
echo mma8452q-trigger > trigger/current_trigger
echo 1 > buffer/enable

```

După activarea bufferului, datele pot fi citite din dispozitivul caracter corespunzător, de exemplu:

În această etapă, achiziția nu mai depinde de citiri manuale ale fișierelor `in_accel_x_raw`, `in_accel_y_raw` și `in_accel_z_raw`. Datele sunt generate de senzor, semnalate prin întrerupere, preluate de driver și introduse în bufferul IIO.

3.9 Eliberarea resurselor la eliminarea driverului

La eliminarea driverului, funcția `remove` este apelată de kernel. Rolul acesteia este să oprească senzorul și să elibereze referințele care nu sunt gestionate automat. În implementare, funcția folosită este:

```

static void mma8452_remove(struct i2c_client *client)
{
    struct iio_dev *indio_dev = i2c_get_clientdata(client);

```

```
struct mma8452_data *data;

if (!indio_dev)
    return;

data = iio_priv(indio_dev);

mma8452_set_active(data, false);

if (indio_dev->trig) {
    iio_trigger_put(indio_dev->trig);
    indio_dev->trig = NULL;
}

dev_info(&client->dev, "removed\n");
}
```

Senzorul este trecut în standby prin apelul `mma8452_set_active(data, false)`. Această operație oprește achiziția de date și reduce activitatea dispozitivului după eliminarea driverului. Referința către trigger este eliberată prin `iio_trigger_put`, deoarece anterior fusese obținută prin `iio_trigger_get`.

Celelalte resurse importante, cum ar fi dispozitivul IIO, triggerul înregistrat, bufferul și întreruperea, sunt gestionate prin funcții de tip `devm_`. Acestea sunt eliberate automat de kernel atunci când dispozitivul este eliminat. Această abordare reduce riscul de scurgeri de memorie și simplifică funcția de curățare.

3.10 Concluzii privind implementarea driverului

Driverul implementat realizează integrarea accelerometrului MMA8452Q în Linux prin subsistemele I2C și Industrial I/O. La nivel de I2C, driverul identifică senzorul prin registrul `WHO_AM_I`, configurează domeniul de măsurare, frecvența de eșantionare și întreruperea data-ready. La nivel de IIO, driverul expune canale pentru axele X, Y și Z, permite citirea valorilor brute, permite modificarea factorului de scalare și a frecvenței de eșantionare și oferă o interfață standard pentru aplicațiile user-space.

Implementarea include atât citirea directă prin `sysfs`, cât și achiziția prin trigger și buffer IIO. Citirea directă este utilă pentru testare și verificări simple, în timp ce bufferul IIO este potrivit pentru achiziții succesive de date, deoarece valorile sunt colectate automat la apariția întreruperilor generate de senzor[20, 14].

Prin această implementare se obține un lanț complet de funcționare: senzorul este declarat prin Device Tree, kernelul asociază dispozitivul cu driverul, driverul configurează senzorul prin I2C, întreruperea data-ready declanșează triggerul IIO, iar valorile măsurate sunt expuse către spațiul utilizatorului prin interfața standard Industrial I/O[20, 14].

Capitolul 4

Configurare și integrare prin Device Tree

Acest capitol prezintă modul în care accelerometrul MMA8452Q a fost descris în Device Tree și modul în care această descriere permite asocierea dispozitivului hardware cu driverul Linux implementat. În sistemele embedded, multe dispozitive nu pot fi descoperite automat de sistemul de operare, motiv pentru care informațiile despre existența și poziția lor pe magistralele hardware trebuie furnizate explicit. Pentru acest scop, Linux utilizează Device Tree, care descrie componentele hardware disponibile pe platformă [14].

În cazul acestei lucrări, senzorul MMA8452Q este conectat la magistrala I2C 1 a plăcii Raspberry Pi 4 Model B, la adresa 0x1c. În plus, pinul de întrerupere al senzorului este conectat la un pin GPIO al plăcii, astfel încât driverul să poată folosi semnalul data-ready pentru achiziția prin trigger și buffer IIO. Pentru ca driverul să fie apelat automat de kernel, dispozitivul a fost descris printr-un overlay Device Tree.

4.1 Descrierea dispozitivului în Device Tree

Pentru integrarea senzorului în sistem a fost creat un overlay Device Tree dedicat. Acesta descrie senzorul MMA8452Q ca dispozitiv conectat pe magistrala i2c1. În forma sursă, overlay-ul utilizat este:

```
/dts-v1/;
/plugin/;

/ {
    compatible = "brcm,bcm2711";

    fragment@0 {
        target = <&i2c1>;

        __overlay__ {
            #address-cells = <1>;
            #size-cells = <0>;
            status = "okay";

            mma8452q@1c {
                compatible = "nxp,mma8452q";
```


4.2 Configurarea întreruperii în Device Tree

Pentru funcționarea achiziției prin buffer IIO, driverul folosește întreruperea data-ready generată de senzor. Această întrerupere este configurată în Device Tree prin proprietățile:

```
interrupt-parent = <&gpio>;
interrupts = <17 8>;
```

Proprietatea `interrupt-parent` indică faptul că sursa de întrerupere este controlerul GPIO al platformei. Proprietatea `interrupts` descrie linia GPIO folosită și tipul de declanșare al întreruperii. În această lucrare a fost utilizat GPIO17.

Valoarea 4 din proprietatea `interrupts=<17 8>` indică modul de declanșare al întreruperii. Pentru această configurație, întreruperea a fost tratată ca o întrerupere activă pe nivel, corespunzătoare comportamentului observat în sistem la citirea fișierului `/proc/interrupts`.

După aplicarea overlay-ului și încărcarea driverului, kernelul transformă descrierea din Device Tree într-un număr de IRQ disponibil în câmpul `client->irq`. În driver, acest număr este folosit la înregistrarea handlerului de întrerupere:

```
ret = devm_request_threaded_irq(&client->dev,
                                client->irq,
                                NULL,
                                mma8452_irq_thread,
                                IRQF_ONESHOT,
                                "mma8452q",
                                indio_dev);
```

Astfel, Device Tree nu configurează direct logica driverului, ci oferă kernelului informația hardware necesară pentru a lega pinul fizic de întrerupere de handlerul implementat în driver.

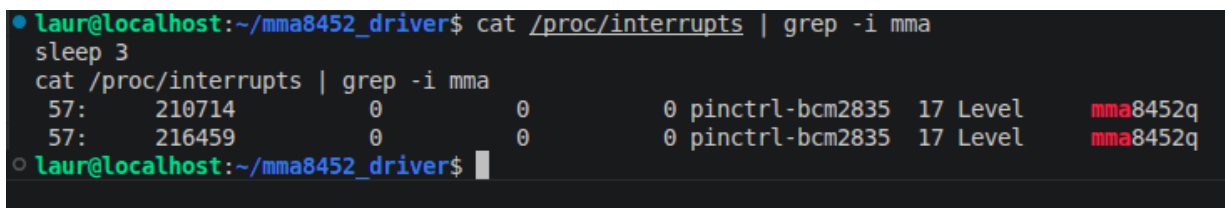
Funcționarea întreruperii poate fi verificată prin comanda:

```
cat /proc/interrupts | grep mma
```

Un rezultat de forma:

```
57: 3779045 0 0 0 pinctrl-bcm2835 17 Level mma8452q
```

arată că întreruperea asociată senzorului este înregistrată și că numărul de întreruperi crește în timpul funcționării. Această verificare confirmă faptul că pinul de întrerupere a fost descris corect în Device Tree și că driverul primește evenimentele generate de senzor.



```
laur@localhost:~/mma8452_driver$ cat /proc/interrupts | grep -i mma
sleep 3
cat /proc/interrupts | grep -i mma
57:    210714          0          0          0 pinctrl-bcm2835 17 Level    mma8452q
57:    216459          0          0          0 pinctrl-bcm2835 17 Level    mma8452q
laur@localhost:~/mma8452_driver$
```

Figura 4.1: Corelarea întreruperii hardware cu triggerul IIO în `/proc/interrupts`

4.3 Asocierea driverului cu dispozitivul

După încărcarea overlay-ului Device Tree, kernelul cunoaște faptul că pe magistrala `i2c1` există un dispozitiv la adresa `0x1c`, compatibil cu `nxp,mma8452q`. Pe baza acestei informații, subsistemul I2C creează o structură de tip `i2c_client`, care reprezintă dispozitivul din punctul de vedere al kernelului.

În driverul implementat, asocierea cu dispozitivul descris în Device Tree este realizată prin tabela:

```
static const struct of_device_id mma8452_of_match[] = {
    { .compatible = "nxp,mma8452q" },
    { }
};
MODULE_DEVICE_TABLE(of, mma8452_of_match);
```

Șirul `compatible` din driver este identic cu șirul `compatible` din Device Tree. Această potrivire permite kernelului să determine că driverul `mma8452q` poate controla dispozitivul definit în overlay.

Driverul este declarat ca driver I2C prin structura:

```
static struct i2c_driver mma8452_driver = {
    .driver = {
        .name = "mma8452q",
        .of_match_table = mma8452_of_match,
    },
    .probe = mma8452_probe,
    .remove = mma8452_remove,
    .id_table = mma8452_id,
};
```

Câmpul `of_match_table` indică lista de dispozitive Device Tree compatibile cu acest driver. Câmpul `probe` indică funcția care va fi apelată atunci când kernelul găsește un dispozitiv compatibil. În această lucrare, funcția `mma8452_probe()` realizează citirea registrului `WHO_AM_I`, inițializarea senzorului, înregistrarea întreruperii, configurarea triggerului și înregistrarea dispozitivului în subsistemul IIO.

Înregistrarea driverului în subsistemul I2C este realizată prin macro-ul:

```
module_i2c_driver(mma8452_driver);
```

Acest macro generează funcțiile de inițializare și ieșire ale modulului kernel. La încărcarea modulului, driverul este înregistrat în subsistemul I2C. Dacă există deja un dispozitiv compatibil descris în Device Tree, kernelul apelează automat funcția `probe`. Astfel, asocierea dintre hardware și driver se face fără ca aplicațiile user-space să intervină direct.

Fluxul de asociere poate fi rezumat astfel:

1. overlay-ul Device Tree descrie senzorul pe magistrala `i2c1`, la adresa `0x1c`;
2. overlay-ul descrie și linia de întrerupere asociată senzorului;
3. kernelul aplică overlay-ul la pornirea sistemului;
4. subsistemul I2C creează un obiect `i2c_client` pentru senzor;

5. driverul se înregistrează în subsistemul I2C;
6. kernelul compară valoarea `compatible` din Device Tree cu tabela `of_match_table` din driver;
7. dacă valorile coincid, este apelată funcția `mma8452_probe()`;
8. driverul identifică senzorul, configurează întreruperea și îl înregistrează în subsistemul IIO.

Această secvență demonstrează rolul Device Tree în integrarea dispozitivelor hardware în kernel. Device Tree nu conține cod de driver, ci doar descrierea hardware necesară pentru ca sistemul de operare să știe ce dispozitive există și unde sunt conectate [14].

4.4 Încărcarea și verificarea driverului în sistem

După configurarea Device Tree și compilarea modulului kernel, driverul poate fi încărcat în sistem. Modulul rezultat în urma compilării are extensia `.ko`. În cazul acestei lucrări, modulul driverului este:

```
mma8452_iio.ko
```

Încărcarea manuală a modulului se poate realiza cu comanda:

```
sudo insmod mma8452_iio.ko
```

Alternativ, după instalarea modulului în directorul corespunzător al kernelului și actualizarea dependențelor, încărcarea se poate face cu:

```
sudo modprobe mma8452_iio
```

Pentru încărcarea automată la pornirea sistemului, numele modulului poate fi adăugat în fișierele de configurare ale sistemului, de exemplu în `/etc/modules` sau într-un fișier din directorul `/etc/modules-load.d/`. În acest mod, driverul este încărcat automat după pornirea sistemului, iar asocierea cu dispozitivul descris în Device Tree se face fără intervenție manuală.

După încărcarea modulului, mesajele generate de driver pot fi verificate cu ajutorul comenzii:

```
dmesg | grep -i mma
```

În cazul unei inițializări corecte, driverul afișează mesaje de forma:

```
mma8452q 1-001c: client irq = 57
mma8452q 1-001c: irq registered: 57
mma8452q 1-001c: MMA8452Q IIO registered with triggered buffer
```

Aceste mesaje indică faptul că funcția `probe` a fost executată cu succes, senzorul a fost identificat corect, întreruperea hardware a fost înregistrată, iar dispozitivul a fost înregistrat în subsistemul IIO cu suport pentru trigger și buffer.

După înregistrarea în subsistemul IIO, dispozitivul poate fi verificat în directorul:

```
/sys/bus/iio/devices/
```

În sistemul testat au fost create atât dispozitivul IIO, cât și triggerul asociat:

```
iio:device0 trigger0
```

Pentru identificarea dispozitivului se poate citi fișierul `name`:

```
cat /sys/bus/iio/devices/iio:device0/name
```

Rezultatul obținut este:

```
mma8452q
```

Pentru identificarea triggerului se poate citi fișierul corespunzător:

```
cat /sys/bus/iio/devices/trigger0/name
```

Rezultatul obținut este:

```
mma8452q-trigger
```

Prezența dispozitivului `iio:device0` confirmă faptul că senzorul a fost înregistrat în sub-sistemul IIO. Prezența triggerului `mma8452q-trigger` confirmă faptul că driverul a creat mecanismul necesar pentru achiziția bazată pe întreruperi.

Valorile brute ale accelerației pot fi citite prin fișierele `sysfs`:

```
cat in_accel_x_raw
cat in_accel_y_raw
cat in_accel_z_raw
```

În timpul testării au fost obținute valori de forma:

```
19
-71
1028
```

Aceste valori corespund orientării senzorului în repaus, unde una dintre axe măsoară aproximativ accelerația gravitațională, iar celelalte două au valori apropiate de zero, cu mici variații datorate zgometului și poziției fizice a modulului.

Parametrii configurabili pot fi verificați prin:

```
cat in_accel_scale
cat in_accel_scale_available
cat in_accel_sampling_frequency
cat in_accel_sampling_frequency_available
```

Exemple de valori disponibile sunt:

Valorile disponibile pentru factorul de scalare sunt:

```
0.000976562 0.001953125 0.003906250
```

Valorile disponibile pentru frecvența de eșantionare sunt:

```
800.000000 400.000000 200.000000 100.000000
50.000000 12.500000 6.250000 1.560000
```

Aceste valori demonstrează că driverul expune către user-space atât domeniile de măsurare disponibile, cât și frecvențele de eșantionare suportate.

Pentru verificarea bufferului IIO, canalele de scanare pot fi activate astfel:

```
cd /sys/bus/iio/devices/iio:device0

echo 0 | sudo tee buffer/enable
echo mma8452q-trigger | sudo tee trigger/current_trigger

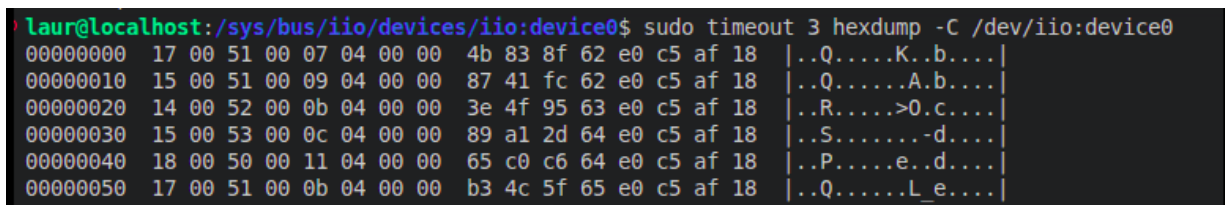
echo 1 | sudo tee scan_elements/in_accel_x_en
echo 1 | sudo tee scan_elements/in_accel_y_en
echo 1 | sudo tee scan_elements/in_accel_z_en
echo 1 | sudo tee scan_elements/in_timestamp_en

echo 16 | sudo tee buffer/length
echo 1 | sudo tee buffer/enable
```

După activarea bufferului, datele pot fi citite din dispozitivul caracter:

```
sudo hexdump -C /dev/iio:device0
```

Apariția unor blocuri succesive de date confirmă faptul că întreruperea hardware declanșează triggerul IIO, iar driverul introduce valorile citite în buffer.



```
laur@localhost:/sys/bus/iio/devices/iio:device0$ sudo timeout 3 hexdump -C /dev/iio:device0
00000000 17 00 51 00 07 04 00 00 4b 83 8f 62 e0 c5 af 18 |..Q.....K..b....|
00000010 15 00 51 00 09 04 00 00 87 41 fc 62 e0 c5 af 18 |..Q.....A.b....|
00000020 14 00 52 00 0b 04 00 00 3e 4f 95 63 e0 c5 af 18 |..R.....>0.c....|
00000030 15 00 53 00 0c 04 00 00 89 a1 2d 64 e0 c5 af 18 |..S.....-d....|
00000040 18 00 50 00 11 04 00 00 65 c0 c6 64 e0 c5 af 18 |..P.....e..d....|
00000050 17 00 51 00 0b 04 00 00 b3 4c 5f 65 e0 c5 af 18 |..Q.....L_e....|
```

Figura 4.2: Citirea datelor brute din bufferul IIO

Prin urmare, integrarea prin Device Tree realizează legătura dintre configurația hardware reală și driverul kernel implementat. Overlay-ul descrie senzorul, adresa sa pe magistrala I2C și linia de întrerupere, iar driverul folosește tabela de compatibilitate pentru a se atașa automat la dispozitiv. Această metodă este specifică sistemelor embedded Linux și permite separarea clară între descrierea hardware și codul driverului.

Capitolul 5

Aplicația user-space și validare experimentală

Acest capitol prezintă aplicația user-space realizată pentru testarea și validarea driverului Linux IIO implementat pentru accelerometrul MMA8452Q. Aplicația a fost dezvoltată în limbajul Python și oferă o interfață grafică realizată cu biblioteca Tkinter, biblioteca standard folosită în Python pentru interfețe grafice bazate pe Tk [21]. Aplicația permite citirea, configurarea și monitorizarea datelor furnizate de senzor.

Scopul aplicației nu este accesarea directă a senzorului prin magistrala I2C, ci utilizarea interfeței standard expuse de subsistemul Industrial I/O. IIO expune dispozitivele prin atribute sysfs și, pentru achiziții cu buffer, prin dispozitive caracter de forma `/dev/iio:deviceX` [10, 12, 22]. Astfel, aplicația verifică faptul că driverul kernel funcționează corect și că datele sunt disponibile în spațiul utilizatorului.

Aplicația permite citirea valorilor brute ale accelerației pe axele X, Y și Z, conversia acestora în unități g, configurarea domeniului de măsurare, configurarea frecvenței de eșantionare, calibrarea de bază prin offset, salvarea datelor în format CSV și citirea datelor prin bufferul IIO[12].

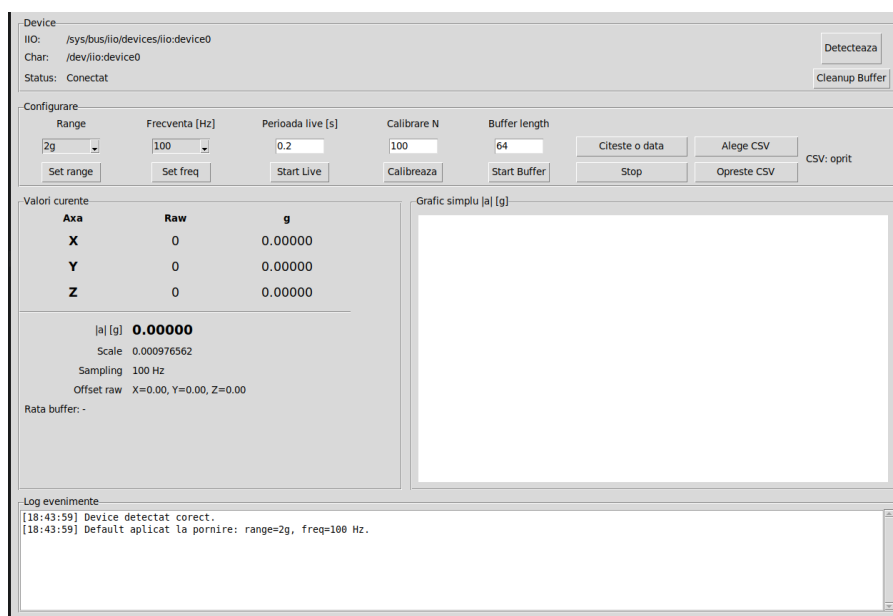


Figura 5.1: Interfața grafică a aplicației user-space pentru testarea accelerometrului

5.1 Interfața user-space pentru accesarea datelor IIO

Driverul implementat expune senzorul prin subsistemul Industrial I/O. În urma încărcării driverului, în sistem apare un dispozitiv de forma `/sys/bus/iio/devices/iio:deviceX` unde `X` este un număr atribuit automat de kernel. Deoarece acest număr poate varia de la o pornire la alta, aplicația nu presupune că senzorul se află mereu în `iio:device0`. În schimb, aplicația caută automat dispozitivul corect citind fișierul `name` pentru fiecare dispozitiv IIO disponibil.

Identificarea dispozitivului se face după valoarea `mma8452q`. Această metodă este mai robustă decât folosirea unei căi fixe, deoarece permite aplicației să funcționeze chiar dacă în sistem apar și alte dispozitive IIO.

Accesul la driver este încapsulat în clasa `MMA8452QDevice`. Aceasta grupează operațiile de citire și scriere în `sysfs`, citirea din bufferul IIO și conversia valorilor brute în unități fizice.

5.2 Citirea valorilor în mod direct prin `sysfs`

Citirea directă a valorilor se realizează prin fișierele `sysfs` create de driver pentru fiecare axă:

```
in_accel_x_raw
in_accel_y_raw
in_accel_z_raw
```

Aceste fișiere conțin valorile brute returnate de senzor pentru axele X, Y și Z. Aplicația citește aceste valori și le transformă în numere întregi.

Un exemplu de valori citite este:

```
19
-71
1028
```

Aceste valori sunt valori brute, nu accelerații exprimate direct în unități fizice. Pentru interpretarea lor este necesar factorul de scalare expus de driver prin atributul:

```
in_accel_scale
```

Conversia într-o valoare exprimată în `g` se face prin relația:

$$a[g] = raw \cdot scale$$

Pentru cele trei axe, aplicația calculează:

$$a_x = x_{raw} \cdot scale$$

$$a_y = y_{raw} \cdot scale$$

$$a_z = z_{raw} \cdot scale$$

Pe lângă valorile individuale ale axelor, aplicația calculează și modulul vectorului accelerație:

$$|a| = \sqrt{a_x^2 + a_y^2 + a_z^2}$$

Această valoare este utilă pentru verificarea comportamentului senzorului în repaus. Când senzorul este nemișcat, modulul accelerației trebuie să fie apropiat de 1g, deoarece senzorul măsoară accelerația gravitațională.

5.3 Citirea datelor prin bufferul IIO

Pe lângă citirea directă prin sysfs, aplicația permite și citirea datelor prin bufferul IIO. Această metodă este mai potrivită pentru achiziții continue, deoarece nucleul IIO oferă suport pentru captură continuă bazată pe trigger, iar mai multe canale pot fi citite împreună din dispozitivul caracter `/dev/iio:deviceX` [12].

Pentru activarea bufferului, aplicația configurează canalele de scanare:

```
scan_elements/in_accel_x_en
scan_elements/in_accel_y_en
scan_elements/in_accel_z_en
scan_elements/in_timestamp_en
```

După activarea canalelor, aplicația selectează triggerul creat de driver:

```
trigger/current_trigger
```

În cazul implementării realizate, triggerul are numele:

```
mma8452q-trigger
```

Apoi se setează dimensiunea bufferului și se activează bufferul:

```
buffer/length
buffer/enable
```

După activarea bufferului, datele sunt citite din dispozitivul caracter:

```
/dev/iio:deviceX
```

Un eșantion citit din buffer conține valorile celor trei axe și timestampul IIO asociat. În aplicație, dimensiunea unui eșantion este de 16 octeți: câte 2 octeți pentru fiecare axă, urmate de aliniere și de un timestamp pe 64 de biți.

Câmp	Dimensiune	Rol
X	2 octeți	valoare brută axa X
Y	2 octeți	valoare brută axa Y
Z	2 octeți	valoare brută axa Z
Padding	2 octeți	aliniere
Timestamp	8 octeți	timp IIO în ns

Tabela 5.1: Structura unui eșantion citit din bufferul IIO

Citirea prin buffer este realizată într-un fir de execuție separat, pentru ca interfața grafică să nu fie blocată. Modulul `threading` din Python permite rularea mai multor fire de execuție în cadrul aceluiasi proces, fiind util în special pentru operații de tip I/O [23, 24]. La frecvențe mari, cum este 800 Hz, aplicația citește continuu datele din buffer, dar nu actualizează interfața grafică pentru fiecare eșantion.

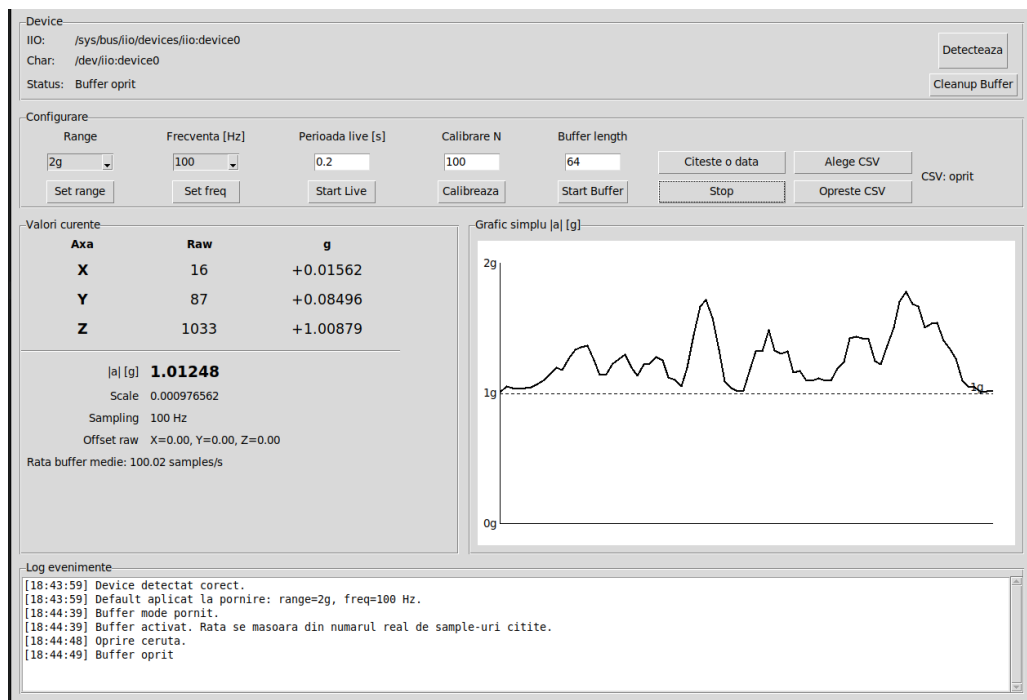


Figura 5.2: Aplicația user-space în modul de achiziție prin buffer IIO

5.4 Configurarea domeniului de măsurare și a frecvenței

Aplicația permite configurarea domeniului de măsurare prin atributul:

`in_accel_scale`

Utilizatorul poate selecta unul dintre cele trei domenii suportate de senzor: $\pm 2g$, $\pm 4g$ și $\pm 8g$, valori definite în documentația tehnică a accelerometrului MMA8452Q [2]. În aplicație, aceste opțiuni sunt mapate către valorile de scalare expuse de driver.

Domeniu selectat	Valoare scrisă în <code>in_accel_scale</code>
$\pm 2g$	0.000976562
$\pm 4g$	0.001953125
$\pm 8g$	0.003906250

Tabela 5.2: Configurarea domeniului de măsurare din aplicație

Frecvența de eșantionare este configurată prin atributul:

`in_accel_sampling_frequency`

Valorile disponibile sunt citite din:

`in_accel_sampling_frequency_available`

În aplicație au fost incluse frecvențele:

1.56 6.25 12.5 50 100 200 400 800

După scrierea unei frecvențe, aplicația citește din nou atributul:

`in_accel_sampling_frequency`

Această citire verifică valoarea aplicată efectiv de driver.

5.5 Calibrare de bază și prelucrarea valorilor

Aplicația include o funcție de calibrare simplă bazată pe offset. În timpul calibrării, senzorul trebuie menținut nemișcat, iar aplicația citește un număr N de eșantioane brute. Pentru fiecare axă se calculează media valorilor citite:

$$offset_x = \frac{1}{N} \sum_{i=1}^N x_i$$

$$offset_y = \frac{1}{N} \sum_{i=1}^N y_i$$

$$offset_z = \frac{1}{N} \sum_{i=1}^N z_i$$

După calcularea offsetului, valorile afișate sunt corectate prin scăderea acestuia:

$$x_{calibrat} = x_{raw} - offset_x$$

$$y_{calibrat} = y_{raw} - offset_y$$

$$z_{calibrat} = z_{raw} - offset_z$$

Această calibrare nu reprezintă o calibrare metrologică completă a senzorului, ci o compensare simplă de offset pentru observații relative. Deoarece senzorul măsoară și accelerația gravitațională, calibrarea prin medie poate elimina și componenta gravitațională de pe axa pe care senzorul este orientat în repaus. Din acest motiv, metoda este utilă pentru testarea variațiilor relative, nu pentru determinarea absolută a orientării.

Pe lângă calibrare, aplicația calculează valorile în g și modulul vectorului accelerație. Aceste rezultate sunt afișate în interfața grafică și pot fi salvate într-un fișier CSV, folosind suportul oferit de biblioteca standard Python pentru acest format [25, 26].

5.6 Testarea funcționării driverului

Testarea driverului a fost realizată în mai multe etape. Prima etapă a constat în verificarea apariției dispozitivului IIO în sistem:

```
ls /sys/bus/iio/devices/
```

În urma încărcării driverului, au fost observate dispozitivul IIO și triggerul asociat:

```
iio:device0 trigger0
```

Identitatea dispozitivului a fost verificată prin citirea fișierului:

```
cat /sys/bus/iio/devices/iio:device0/name
```

Rezultatul obținut a fost:

```
mma8452q
```

Apoi au fost testate citirile directe:

```
cat in_accel_x_raw
cat in_accel_y_raw
cat in_accel_z_raw
```

Pentru configurare, au fost verificate attributele:

```
cat in_accel_scale
cat in_accel_scale_available
cat in_accel_sampling_frequency
cat in_accel_sampling_frequency_available
```

Testarea bufferului a fost realizată prin activarea canalelor de scanare și prin citirea datelor din:

```
/dev/iio:device0
```

Un exemplu de citire prin buffer a fost realizat cu:

```
sudo hexdump -C /dev/iio:device0
```

Apariția continuă a datelor în `hexdump` confirmă faptul că bufferul IIO primește eșantioane. În plus, în `/proc/interrupts` s-a observat creșterea numărului de întreruperi pentru senzor și apariția triggerului asociat.

```
cat /proc/interrupts | grep mma
```

Un rezultat reprezentativ este:

```
57: 3779045 0 0 0 pinctrl-bcm2835 17 Level mma8452q
58: 0 0 306 0 mma8452q-trigger Edge mma8452q_consumer0
```

Acest rezultat arată că întreruperea hardware a senzorului este activă și că triggerul IIO este folosit de buffer.

5.7 Validarea experimentală a rezultatelor

Validarea experimentală a fost realizată prin compararea comportamentului senzorului în mai multe situații. În repaus, una dintre axe trebuie să indice o valoare apropiată de $\pm 1g$, în funcție de orientarea fizică a modului, iar celelalte axe trebuie să aibă valori apropiate de zero.

Un exemplu de valori brute citite în repaus este:

```
X = 19
Y = -71
Z = 1028
```

Pentru domeniul $\pm 2g$, factorul de scalare este:

0.000976562

Aplicând conversia, se obțin valori aproximative:

$$a_x \approx 19 \cdot 0.000976562 = 0.0185g$$

$$a_y \approx -71 \cdot 0.000976562 = -0.0693g$$

$$a_z \approx 1028 \cdot 0.000976562 = 1.0039g$$

Aceste rezultate sunt coerente cu poziția senzorului în repaus, deoarece axa Z măsoară aproximativ accelerația gravitațională. Modulul vectorului accelerație este apropiat de $1g$, ceea ce confirmă faptul că valorile brute și factorul de scalare sunt interpretate corect.

A fost verificată și modificarea domeniului de măsurare prin scrierea în `in_accel_scale`. Valorile citite după modificare au confirmat că driverul acceptă și aplică domeniile $\pm 2g$, $\pm 4g$ și $\pm 8g$.

De asemenea, a fost verificată modificarea frecvenței de eșantionare. Aplicația a putut seta valori precum 50 Hz și 100 Hz.

Valoarea aplicată a fost confirmată prin citirea atributului:

`in_accel_sampling_frequency`

Pentru modul buffer, aplicația a măsurat rata reală de citire a eșantioanelor. Această verificare este importantă deoarece demonstrează că achiziția nu se bazează doar pe citiri manuale din sysfs, ci poate funcționa și în regim continuu, folosind întreruperea hardware și triggerul IIO.

5.8 Impactul asupra resurselor și limitele implementării

Aplicația user-space a fost proiectată astfel încât să nu blocheze interfața grafică în timpul achiziției. Pentru operațiile continue, precum modul live sau modul buffer, aplicația folosește fire de execuție separate. Astfel, citirea datelor se face în fundalul aplicației, iar interfața grafică rămâne utilizabilă.

În modul buffer, aplicația nu actualizează interfața grafică pentru fiecare eșantion citit. Această alegere este necesară deoarece, la frecvențe mari de eșantionare, actualizarea grafică pentru fiecare probă ar consuma inutil resurse și ar putea face interfața lentă. În schimb, aplicația citește toate probele disponibile, dar actualizează afișarea la o rată mai mică.

Logging-ul CSV poate avea impact asupra performanței, mai ales la frecvențe ridicate. Pentru a reduce acest impact, aplicația nu face operație de `flush` la fiecare eșantion în modul buffer, ci periodic, după mai multe probe.

Implementarea are și câteva limite. Calibrarea realizată este una simplă, bazată pe offset, și nu înlocuiește o calibrare completă a senzorului. De asemenea, aplicația presupune o structură fixă a eșantionului din buffer, corespunzătoare canalelor activate în driver: X, Y, Z și timestamp. Dacă structura canalelor IIO ar fi modificată în driver, funcția de decodare din aplicație ar trebui actualizată.

O altă limitare este legată de drepturile de acces. Citirea prin sysfs poate fi realizată de obicei fără privilegii speciale, însă citirea din `/dev/iio:deviceX` poate necesita rularea aplicației cu drepturi de administrator sau modificarea permisiunilor dispozitivului.

În concluzie, aplicația user-space demonstrează funcționarea completă a driverului: dispozitivul este detectat prin IIO, valorile brute sunt citite corect, parametrii senzorului pot fi modificați, bufferul IIO funcționează, iar datele pot fi prelucrate și salvate în format CSV. Astfel, aplicația confirmă integrarea completă a accelerometrului MMA8452Q în sistemul Linux embedded.

Concluzii

Analiza rezultatelor obținute

Lucrarea a avut ca obiectiv proiectarea, implementarea și testarea unui driver Linux pentru accelerometrul triaxial MMA8452Q, integrat în subsistemul Industrial I/O. Driverul a fost dezvoltat pentru o platformă embedded bazată pe Raspberry Pi 4 Model B, folosind comunicația I2C pentru accesarea senzorului și interfața IIO pentru expunerea datelor către spațiul utilizatorului.

În prima etapă a lucrării a fost realizată analiza platformei hardware și software. Senzorul MMA8452Q a fost conectat la magistrala I2C a plăcii Raspberry Pi, iar comunicarea a fost verificată prin utilitare user-space precum `i2cdetect` și `i2cget`. Detectarea dispozitivului la adresa `0x1c` și citirea valorii `0x2A` din registrul `WHO_AM_I` au confirmat faptul că senzorul este conectat corect și poate comunica prin magistrala I2C.

În etapa de implementare a fost realizat un driver kernel în limbajul C, structurat ca driver I2C și integrat în subsistemul IIO. Driverul identifică senzorul prin registrul `WHO_AM_I`, configurează domeniul de măsurare, configurează frecvența de eșantionare și permite citirea valorilor brute ale accelerației pe axele X, Y și Z. De asemenea, driverul expune către user-space atribute standard IIO, precum `in_accel_x_raw`, `in_accel_y_raw`, `in_accel_z_raw`, `in_accel_scale` și `in_accel_sampling_frequency`.

Integrarea dispozitivului în sistem a fost realizată prin Device Tree. A fost creat un overlay Device Tree care descrie senzorul pe magistrala `i2c1`, la adresa `0x1c`, folosind proprietatea `compatible = "nxp,mma8452q"`. Această descriere permite kernelului să asocieze automat dispozitivul hardware cu driverul implementat. Prin această metodă, driverul nu trebuie legat manual de o adresă I2C, ci este apelat automat prin mecanismul standard de detecție al kernelului.

Rezultatele obținute în urma testării confirmă funcționarea corectă a sistemului implementat. După încărcarea driverului, dispozitivul a fost înregistrat în subsistemul IIO și a apărut în directorul `/sys/bus/iio/devices/`. Citirea fișierului `name` a confirmat identificarea dispozitivului ca `mma8452q`. Valorile brute ale accelerației au putut fi citite pentru toate cele trei axe, iar modificarea domeniului de măsurare și a frecvenței de eșantionare a fost realizată cu succes prin atributele sysfs expuse de driver.

În timpul testării, s-a observat că valorile citite de senzor sunt coerente cu orientarea fizică a modului. În repaus, una dintre axe indică o valoare apropiată de $\pm 1g$, în timp ce celelalte axe au valori apropiate de zero. Acest comportament este așteptat, deoarece accelerometrul măsoară accelerația gravitațională. Conversia valorilor brute folosind factorul de scalare expus prin `in_accel_scale` a permis obținerea unor valori exprimate în unități `g`, utile pentru interpretarea rezultatelor.

Aplicația Python a fost folosită pentru validarea driverului. Funcțiile principale sunt:

- detectarea automată a dispozitivului IIO;
- citirea valorilor brute pentru axele X, Y și Z;
- conversia valorilor brute în unități g;
- calcularea modulului vectorului accelerație;
- configurarea domeniului de măsurare;
- configurarea frecvenței de eșantionare;
- salvarea datelor în format CSV.

Prin această aplicație s-a demonstrat că driverul poate fi utilizat de programe user-space fără acces direct la registrele senzorului.

Un rezultat important al lucrării este separarea clară dintre nivelul hardware, nivelul kernel și nivelul user-space. Comunicația directă cu senzorul este realizată în driver, integrarea în sistem este realizată prin Device Tree, iar aplicația user-space folosește doar interfața standard IIO. Această organizare respectă modelul folosit în sistemele Linux embedded și face implementarea mai ușor de extins și de întreținut.

În urma implementării, se poate considera că obiectivele principale ale lucrării au fost atinse. Senzorul este detectat corect, driverul se atașează automat prin Device Tree, datele sunt citite prin I2C, valorile sunt expuse prin IIO, iar aplicația user-space poate configura și monitoriza funcționarea senzorului. Sistemul rezultat poate fi folosit ca bază pentru aplicații simple de monitorizare a accelerației, detecție a orientării sau achiziție de date de mișcare.

Opinia personală

Din punct de vedere personal, lucrarea a fost utilă deoarece a permis înțelegerea practică a modului în care un dispozitiv hardware este integrat într-un sistem Linux. Implementarea unui driver kernel a evidențiat diferența dintre programarea obișnuită în user-space și dezvoltarea la nivel de kernel, unde trebuie respectate reguli stricte privind accesul la resurse, sincronizarea, tratarea erorilor și interacțiunea cu subsistemele existente.

Un aspect important al lucrării a fost faptul că implementarea nu s-a limitat la citirea unor valori de la senzor, ci a urmărit integrarea completă a acestuia în arhitectura Linux. Pentru ca sistemul să funcționeze corect, a fost necesară înțelegerea relației dintre Device Tree, subsistemul I2C, funcția `probe`, structura `i2c_client`, subsistemul IIO și aplicația user-space. Această legătură dintre componente a reprezentat partea cea mai importantă a lucrării, deoarece arată modul real în care un dispozitiv hardware ajunge să fie accesibil într-un sistem Linux embedded.

De asemenea, lucrul cu IIO a fost relevant deoarece oferă o interfață standard pentru senzori. În locul unei soluții improvizate, bazate pe fișiere proprii sau pe acces direct la magistrala I2C din user-space, driverul folosește mecanismele existente ale kernelului. Din acest motiv, soluția este mai apropiată de modul în care sunt implementate driverele reale din Linux.

Pe parcursul lucrării au apărut și dificultăți, în special în înțelegerea ordinii de inițializare a driverului, a modului în care Device Tree creează dispozitivul I2C și a modului în care IIO expune automat atributele în `sysfs`[27]. Aceste dificultăți au fost utile, deoarece au clarificat diferența dintre existența fizică a senzorului pe magistrală și existența lui ca dispozitiv gestionat de kernel.

Consider că lucrarea a avut un caracter practic puternic, deoarece rezultatul final poate fi testat direct pe hardware real. Faptul că senzorul este detectat, configurat și citit printr-un driver kernel propriu confirmă că implementarea nu este doar teoretică, ci funcțională într-un sistem embedded Linux.

Limitele implementării

Există însă și limite ale implementării. Driverul realizat se concentrează pe funcțiile principale ale senzorului: identificare, configurare de bază și citire de date. Nu au fost implementate toate funcțiile avansate disponibile în MMA8452Q, cum ar fi detecția de mișcare, detecția de cădere liberă, detecția de orientare, modurile avansate de consum redus sau filtrarea internă a datelor.

De asemenea, calibrarea realizată în aplicația user-space este una simplă, bazată pe offset, și nu reprezintă o calibrare completă a senzorului. Pentru aplicații care necesită precizie ridicată, ar fi necesară o calibrare mai complexă, care să ia în considerare erorile de scalare, deviațiile între axe și eventualele influențe mecanice ale montajului.

O altă limitare este legată de faptul că partea de buffer și trigger IIO poate fi extinsă. Deși aplicația user-space include suport pentru citirea din buffer, implementarea poate fi dezvoltată în continuare pentru o integrare mai completă cu mecanismele IIO de achiziție continuă. În forma actuală, driverul este potrivit pentru validarea funcțiilor principale și pentru achiziții simple, dar nu acoperă toate scenariile posibile de utilizare intensivă a senzorului.

În plus, testarea experimentală a fost realizată în condiții de laborator, prin observarea valorilor brute, a valorilor convertite în g și a comportamentului senzorului la schimbarea orientării. Pentru o validare metrologică completă ar fi necesare echipamente de referință și teste controlate, prin care valorile măsurate să fie comparate cu valori cunoscute ale accelerației.

Perspective viitoare de dezvoltare

Ca perspective viitoare de dezvoltare, driverul poate fi extins prin implementarea completă a funcțiilor de întrerupere ale senzorului. MMA8452Q permite generarea de întreruperi pentru mai multe evenimente interne, iar aceste funcții ar putea fi integrate în driver pentru a permite aplicații mai eficiente, bazate pe evenimente, nu doar pe citire periodică.

O altă direcție de dezvoltare este extinderea suportului pentru bufferul IIO și pentru triggerul hardware. Prin utilizarea completă a bufferelor IIO, sistemul poate realiza achiziții continue la frecvențe mai mari, cu o încărcare mai redusă asupra aplicației user-space. Această direcție este importantă pentru aplicații în care este necesară înregistrarea continuă a mișcării sau analiza vibrațiilor.

De asemenea, aplicația user-space poate fi îmbunătățită prin adăugarea unor funcții de vizualizare mai avansate. De exemplu, se pot adăuga grafice separate pentru axele X, Y și Z, export automat al sesiunilor de testare, calculul unor indicatori statistici și detectarea unor evenimente simple pe baza pragurilor de accelerație.

O altă perspectivă este integrarea sistemului într-o aplicație practică. Driverul și aplicația pot fi folosite ca bază pentru un sistem de monitorizare a vibrațiilor, un sistem de detecție a poziției, un sistem de înregistrare a mișcării sau o aplicație educațională pentru studiul senzorilor MEMS în Linux embedded.

În concluzie, lucrarea demonstrează integrarea funcțională a unui accelerometru digital într-un sistem Linux embedded, pornind de la conectarea fizică a senzorului și până la accesarea datelor dintr-o aplicație user-space. Implementarea realizată confirmă funcționarea lanțului complet hardware–kernel–user-space și oferă o bază solidă pentru dezvoltări ulterioare în domeniul driverelor Linux pentru senzori.

Codurile sursă dezvoltate în cadrul lucrării sunt prezentate în Anexa 1 – Cod sursă, începând cu pagina 54.

Bibliografie

- [1] Raspberry Pi Ltd. config.txt - Raspberry Pi Documentation. https://www.raspberrypi.com/documentation/computers/config_txt.html, 2026. Accesat la data de: 2026-05-13.
- [2] NXP Semiconductors. MMA8452Q: 3-axis, 12-bit/8-bit digital accelerometer. <https://www.nxp.com/docs/en/data-sheet/MMA8452Q.pdf>, 2017. Accesat la data de: 2026-05-13.
- [3] NXP Semiconductors. Data Manipulation and Basic Settings of the MMA8451, 2, 3Q. <https://www.nxp.com/docs/en/application-note/AN4076.pdf>, 2011. Accesat la data de: 2026-05-13.
- [4] The Linux Kernel Organization. Introduction to I2C and SMBus. <https://docs.kernel.org/i2c/summary.html>, 2026. Accesat la data de: 2026-05-13.
- [5] Raspberry Pi Ltd. Raspberry Pi computer hardware. <https://www.raspberrypi.com/documentation/computers/raspberry-pi.html>, 2026. Accesat la data de: 2026-05-13.
- [6] The Linux Kernel Organization. The I2C Protocol. <https://docs.kernel.org/i2c/i2c-protocol.html>, 2026. Accesat la data de: 2026-05-13.
- [7] The Linux Kernel Organization. Device drivers infrastructure. <https://docs.kernel.org/driver-api/infrastructure.html>, 2026. Accesat la data de: 2026-05-13.
- [8] The Linux Kernel Organization. Mutexes in Linux. <https://docs.kernel.org/locking/mutex-design.html>, 2026. Accesat la data de: 2026-05-13.
- [9] The Linux Kernel Organization. Industrial I/O. <https://docs.kernel.org/driver-api/iio/index.html>, 2026. Accesat la data de: 2026-05-13.
- [10] The Linux Kernel Organization. Industrial I/O Core elements. <https://docs.kernel.org/driver-api/iio/core.html>, 2026. Accesat la data de: 2026-05-13.
- [11] The Linux Kernel Organization. The Linux kernel user-space API guide. <https://www.kernel.org/doc/html/v6.6/userspace-api/index.html>, 2026. Accesat la data de: 2026-05-13.
- [12] The Linux Kernel Organization. Industrial I/O device buffers. https://docs.kernel.org/iio/iio_devbuf.html, 2026. Accesat la data de: 2026-05-13.

- [13] The Linux Kernel Organization. Industrial I/O triggers. <https://docs.kernel.org/driver-api/iio/triggers.html>, 2026. Accesat la data de: 2026-05-13.
- [14] The Linux Kernel Organization. Linux and the Devicetree. <https://docs.kernel.org/devicetree/usage-model.html>, 2026. Accesat la data de: 2026-05-13.
- [15] The Linux Kernel Organization. How to instantiate I2C devices. <https://docs.kernel.org/6.2/i2c/instantiating-devices.html>, 2023. Accesat la data de: 2026-05-13.
- [16] Raspberry Pi Ltd. Device Trees, overlays, and parameters. <https://www.raspberrypi.com/documentation/computers/configuration.html#device-trees-overlays-and-parameters>, 2026. Accesat la data de: 2026-05-13.
- [17] The Linux Kernel Organization. Introduction to Industrial I/O. <https://docs.kernel.org/driver-api/iio/intro.html>, 2026. Accesat la data de: 2026-05-13.
- [18] The Linux Kernel Organization. IIO triggered buffer setup. <https://docs.kernel.org/driver-api/iio/triggered-buffers.html>, 2026. Accesat la data de: 2026-05-13.
- [19] The Linux Kernel Organization. IIO buffer setup. <https://docs.kernel.org/driver-api/iio/buffers.html>, 2026. Accesat la data de: 2026-05-13.
- [20] The Linux Kernel Organization. Generic IRQ handling. <https://docs.kernel.org/core-api/genericirq.html>, 2026. Accesat la data de: 2026-05-13.
- [21] Python Software Foundation. tkinter — Python interface to Tcl/Tk. <https://docs.python.org/3/library/tkinter.html>, 2026. Accesat la data de: 2026-05-13.
- [22] The Linux Kernel Organization. IIO sysfs ABI documentation. <https://docs.kernel.org/admin-guide/abi-testing-files.html>, 2026. Accesat la data de: 2026-05-13.
- [23] Python Software Foundation. threading — Thread-based parallelism. <https://docs.python.org/3/library/threading.html>, 2026. Accesat la data de: 2026-05-13.
- [24] Python Software Foundation. struct — Interpret bytes as packed binary data. <https://docs.python.org/3/library/struct.html>, 2026. Accesat la data de: 2026-05-13.
- [25] Python Software Foundation. csv — CSV File Reading and Writing. <https://docs.python.org/3/library/csv.html>, 2026. Accesat la data de: 2026-05-13.
- [26] Python Software Foundation. pathlib — Object-oriented filesystem paths. <https://docs.python.org/3/library/pathlib.html>, 2026. Accesat la data de: 2026-05-13.
- [27] The Linux Kernel Organization. sysfs - The filesystem for exporting kernel objects. <https://docs.kernel.org/filesystems/sysfs.html>, 2026. Accesat la data de: 2026-05-13.

Anexă 1 - Codul sursă

.1 Driver Linux IIO pentru MMA8452Q

```

1 #include <linux/module.h>
2 #include <linux/i2c.h>
3 #include <linux/of.h>
4 #include <linux/mutex.h>
5 #include <linux/interrupt.h>
6 #include <linux/bitops.h>
7 #include <linux/string.h>
8
9 #include <linux/iio/trigger_consumer.h>
10 #include <linux/iio/buffer.h>
11 #include <linux/iio/iio.h>
12 #include <linux/iio/trigger.h>
13 #include <linux/iio/triggered_buffer.h>
14
15 #define MMA8452_STATUS                0x00
16 #define MMA8452_OUT_X_MSB             0x01
17 #define MMA8452_WHO_AM_I              0x0D
18 #define MMA8452_INT_SOURCE            0x0C
19 #define MMA8452_XYZ_DATA_CFG          0x0E
20 #define MMA8452_CTRL_REG1             0x2A
21 #define MMA8452_CTRL_REG4             0x2D
22 #define MMA8452_CTRL_REG5             0x2E
23
24 #define MMA8452_WHO_AM_I_VAL          0x2A
25
26 #define MMA8452_CTRL_REG1_ACTIVE      BIT(0)
27 #define MMA8452_CTRL_REG1_DR_SHIFT   3
28 #define MMA8452_CTRL_REG1_DR_MASK    GENMASK(5, 3)
29
30 #define MMA8452_INT_DRDY               BIT(0)
31
32 static const int mma8452_scale_nano[] = {
33     976562,
34     1953125,
35     3906250,
36 };
37
38 static const int mma8452_odr_millihz[] = {
39     800000,
40     400000,
41     200000,
42     100000,
43     50000,
44     12500,
45     6250,
46     1560,
47 };
48
49 struct mma8452_data {
50     struct i2c_client *client;
51     struct mutex lock;
52     struct iio_trigger *trig;

```

```

53     u8 fs_bits;
54     u8 odr_bits;
55 };
56
57 struct mma8452_scan {
58     s16 channels[3];
59     s64 timestamp __aligned(8);
60 };
61
62 static s16 mma8452_convert_12bit(u8 msb, u8 lsb)
63 {
64     u16 raw = ((u16)msb << 8) | lsb;
65
66     return sign_extend32(raw >> 4, 11);
67 }
68
69 static int mma8452_set_active(struct mma8452_data *data, bool active)
70 {
71     int ctrl;
72
73     ctrl = i2c_smbus_read_byte_data(data->client, MMA8452_CTRL_REG1);
74     if (ctrl < 0)
75         return ctrl;
76
77     if (active)
78         ctrl |= MMA8452_CTRL_REG1_ACTIVE;
79     else
80         ctrl &= ~MMA8452_CTRL_REG1_ACTIVE;
81
82     return i2c_smbus_write_byte_data(data->client, MMA8452_CTRL_REG1, ctrl);
83 }
84
85 static int mma8452_set_range(struct mma8452_data *data, u8 fs_bits)
86 {
87     int ret;
88
89     if (fs_bits > 2)
90         return -EINVAL;
91
92     ret = mma8452_set_active(data, false);
93     if (ret < 0)
94         return ret;
95
96     ret = i2c_smbus_write_byte_data(data->client,
97                                     MMA8452_XYZ_DATA_CFG,
98                                     fs_bits & 0x03);
99     if (ret < 0)
100         goto reactivate;
101
102     data->fs_bits = fs_bits;
103
104 reactivate:
105     mma8452_set_active(data, true);
106     return ret;
107 }
108
109 static int mma8452_set_odr(struct mma8452_data *data, u8 odr_bits)
110 {
111     int ctrl;
112     int ret;
113
114     if (odr_bits > 7)
115         return -EINVAL;
116
117     ret = mma8452_set_active(data, false);
118     if (ret < 0)
119         return ret;
120
121     ctrl = i2c_smbus_read_byte_data(data->client, MMA8452_CTRL_REG1);
122     if (ctrl < 0) {
123         ret = ctrl;
124         goto reactivate;
125     }
126
127     ctrl &= ~MMA8452_CTRL_REG1_DR_MASK;
128     ctrl |= (odr_bits << MMA8452_CTRL_REG1_DR_SHIFT) &

```

```

129     MMA8452_CTRL_REG1_DR_MASK;
130
131     ret = i2c_smbus_write_byte_data(data->client, MMA8452_CTRL_REG1, ctrl);
132     if (ret < 0)
133         goto reactivate;
134
135     data->odr_bits = odr_bits;
136
137 reactivate:
138     mma8452_set_active(data, true);
139     return ret;
140 }
141
142 static int mma8452_read_axis(struct mma8452_data *data, int axis, int *val)
143 {
144     int ret;
145     u8 buf[2];
146     u8 reg = MMA8452_OUT_X_MSB + axis * 2;
147
148     ret = i2c_smbus_read_i2c_block_data(data->client, reg, sizeof(buf), buf);
149     if (ret < 0)
150         return ret;
151     if (ret != sizeof(buf))
152         return -EIO;
153
154     *val = mma8452_convert_12bit(buf[0], buf[1]);
155
156     return 0;
157 }
158
159 static int mma8452_read_xyz(struct mma8452_data *data, s16 *x, s16 *y, s16 *z)
160 {
161     int ret;
162     u8 buf[6];
163
164     ret = i2c_smbus_read_i2c_block_data(data->client,
165                                         MMA8452_OUT_X_MSB,
166                                         sizeof(buf),
167                                         buf);
168     if (ret < 0)
169         return ret;
170     if (ret != sizeof(buf))
171         return -EIO;
172
173     *x = mma8452_convert_12bit(buf[0], buf[1]);
174     *y = mma8452_convert_12bit(buf[2], buf[3]);
175     *z = mma8452_convert_12bit(buf[4], buf[5]);
176
177     return 0;
178 }
179
180 static int mma8452_read_raw(struct iio_dev *indio_dev,
181                             const struct iio_chan_spec *chan,
182                             int *val, int *val2, long mask)
183 {
184     struct mma8452_data *data = iio_priv(indio_dev);
185     int ret;
186
187     switch (mask) {
188     case IIO_CHAN_INFO_RAW:
189         mutex_lock(&data->lock);
190         ret = mma8452_read_axis(data, chan->address, val);
191         mutex_unlock(&data->lock);
192
193         if (ret < 0)
194             return ret;
195
196         return IIO_VAL_INT;
197
198     case IIO_CHAN_INFO_SCALE:
199         *val = 0;
200         *val2 = mma8452_scale_nano[data->fs_bits];
201         return IIO_VAL_INT_PLUS_NANO;
202
203     case IIO_CHAN_INFO_SAMP_FREQ:
204         *val = mma8452_odr_millihz[data->odr_bits] / 1000;

```



```

205     *val2 = (mma8452_odr_millihz[data->odr_bits] % 1000) * 1000;
206     return IIO_VAL_INT_PLUS_MICRO;
207
208     default:
209         return -EINVAL;
210 }
211 }
212
213 static int mma8452_write_raw(struct iio_dev *indio_dev,
214                             const struct iio_chan_spec *chan,
215                             int val, int val2, long mask)
216 {
217     struct mma8452_data *data = iio_priv(indio_dev);
218     int target_millihz;
219     int i;
220     int ret = -EINVAL;
221
222     mutex_lock(&data->lock);
223
224     switch (mask) {
225     case IIO_CHAN_INFO_SCALE:
226         if (val != 0)
227             break;
228
229         for (i = 0; i < ARRAY_SIZE(mma8452_scale_nano); i++) {
230             if (val2 == mma8452_scale_nano[i]) {
231                 ret = mma8452_set_range(data, i);
232                 break;
233             }
234         }
235         break;
236
237     case IIO_CHAN_INFO_SAMP_FREQ:
238         target_millihz = val * 1000 + val2 / 1000;
239
240         for (i = 0; i < ARRAY_SIZE(mma8452_odr_millihz); i++) {
241             if (target_millihz == mma8452_odr_millihz[i]) {
242                 ret = mma8452_set_odr(data, i);
243                 break;
244             }
245         }
246         break;
247
248     default:
249         break;
250 }
251
252     mutex_unlock(&data->lock);
253
254     return ret;
255 }
256
257 static int mma8452_read_avail(struct iio_dev *indio_dev,
258                              const struct iio_chan_spec *chan,
259                              const int **vals, int *type, int *length,
260                              long mask)
261 {
262     static const int scale_avail[] = {
263         0, 976562,
264         0, 1953125,
265         0, 3906250,
266     };
267
268     static const int samp_freq_avail[] = {
269         800, 0,
270         400, 0,
271         200, 0,
272         100, 0,
273         50, 0,
274         12, 500000,
275         6, 250000,
276         1, 560000,
277     };
278
279     switch (mask) {
280     case IIO_CHAN_INFO_SCALE:

```

```

281     *vals = scale_avail;
282     *type = IIO_VAL_INT_PLUS_NANO;
283     *length = ARRAY_SIZE(scale_avail);
284     return IIO_AVAIL_LIST;
285
286 case IIO_CHAN_INFO_SAMP_FREQ:
287     *vals = samp_freq_avail;
288     *type = IIO_VAL_INT_PLUS_MICRO;
289     *length = ARRAY_SIZE(samp_freq_avail);
290     return IIO_AVAIL_LIST;
291
292 default:
293     return -EINVAL;
294 }
295 }
296
297 static int mma8452_write_raw_get_fmt(struct iio_dev *indio_dev,
298                                     const struct iio_chan_spec *chan,
299                                     long mask)
300 {
301     switch (mask) {
302     case IIO_CHAN_INFO_SCALE:
303         return IIO_VAL_INT_PLUS_NANO;
304
305     case IIO_CHAN_INFO_SAMP_FREQ:
306         return IIO_VAL_INT_PLUS_MICRO;
307
308     default:
309         return -EINVAL;
310     }
311 }
312
313 static irqreturn_t mma8452_trigger_handler(int irq, void *p)
314 {
315     struct iio_poll_func *pf = p;
316     struct iio_dev *indio_dev = pf->indio_dev;
317     struct mma8452_data *data = iio_priv(indio_dev);
318     struct mma8452_scan scan;
319     int ret;
320
321     memset(&scan, 0, sizeof(scan));
322
323     mutex_lock(&data->lock);
324     ret = mma8452_read_xyz(data,
325                             &scan.channels[0],
326                             &scan.channels[1],
327                             &scan.channels[2]);
328     mutex_unlock(&data->lock);
329
330     if (ret == 0)
331         iio_push_to_buffers_with_timestamp(indio_dev,
332                                             &scan,
333                                             pf->timestamp);
334
335     iio_trigger_notify_done(data->trig);
336
337     return IRQ_HANDLED;
338 }
339
340 static const struct iio_info mma8452_iio_info = {
341     .read_raw = mma8452_read_raw,
342     .write_raw = mma8452_write_raw,
343     .write_raw_get_fmt = mma8452_write_raw_get_fmt,
344     .read_avail = mma8452_read_avail,
345 };
346
347 static const struct iio_trigger_ops mma8452_trigger_ops = {
348     .validate_device = iio_trigger_validate_own_device,
349 };
350
351 #define MMA8452_ACCEL_CHAN(_axis, _addr) \
352 { \
353     .type = IIO_ACCEL, \
354     .modified = 1, \
355     .channel2 = IIO_MOD_##_axis, \
356     .address = _addr, \

```

```

357     .info_mask_separate = BIT(IIO_CHAN_INFO_RAW),    \
358     .info_mask_shared_by_type =                      \
359         BIT(IIO_CHAN_INFO_SCALE) |                  \
360         BIT(IIO_CHAN_INFO_SAMP_FREQ),                \
361     .info_mask_shared_by_type_available =            \
362         BIT(IIO_CHAN_INFO_SCALE) |                  \
363         BIT(IIO_CHAN_INFO_SAMP_FREQ),                \
364     .scan_index = _addr,                             \
365     .scan_type = {                                    \
366         .sign = 's',                                \
367         .realbits = 12,                             \
368         .storagebits = 16,                          \
369         .shift = 0,                                  \
370         .endianness = IIO_CPU,                      \
371     },
372 }
373
374 static const unsigned long mma8452_scan_masks[] = {
375     BIT(0) | BIT(1) | BIT(2),
376     0
377 };
378
379 static const struct iio_chan_spec mma8452_channels[] = {
380     MMA8452_ACCEL_CHAN(X, 0),
381     MMA8452_ACCEL_CHAN(Y, 1),
382     MMA8452_ACCEL_CHAN(Z, 2),
383     IIO_CHAN_SOFT_TIMESTAMP(3),
384 };
385
386 static int mma8452_chip_init(struct mma8452_data *data)
387 {
388     int ret;
389
390     ret = i2c_smbus_write_byte_data(data->client, MMA8452_CTRL_REG1, 0x00);
391     if (ret < 0)
392         return ret;
393
394     ret = i2c_smbus_write_byte_data(data->client, MMA8452_XYZ_DATA_CFG, 0x00);
395     if (ret < 0)
396         return ret;
397
398     ret = i2c_smbus_write_byte_data(data->client,
399                                     MMA8452_CTRL_REG4,
400                                     MMA8452_INT_DRDY);
401     if (ret < 0)
402         return ret;
403
404     ret = i2c_smbus_write_byte_data(data->client,
405                                     MMA8452_CTRL_REG5,
406                                     MMA8452_INT_DRDY);
407     if (ret < 0)
408         return ret;
409
410     ret = i2c_smbus_write_byte_data(data->client,
411                                     MMA8452_CTRL_REG1,
412                                     (3 << MMA8452_CTRL_REG1_DR_SHIFT) |
413                                     MMA8452_CTRL_REG1_ACTIVE);
414     if (ret < 0)
415         return ret;
416
417     data->fs_bits = 0;
418     data->odr_bits = 3;
419
420     return 0;
421 }
422
423 static irqreturn_t mma8452_irq_thread(int irq, void *dev_id)
424 {
425     struct iio_dev *indio_dev = dev_id;
426     struct mma8452_data *data = iio_priv(indio_dev);
427     int source;
428
429     mutex_lock(&data->lock);
430     source = i2c_smbus_read_byte_data(data->client, MMA8452_INT_SOURCE);
431     mutex_unlock(&data->lock);
432

```

```

433     if (source < 0)
434         return IRQ_HANDLED;
435
436     if ((source & MMA8452_INT_DRDY) && data->trig)
437         iio_trigger_poll(data->trig);
438
439     return IRQ_HANDLED;
440 }
441
442 static int mma8452_probe(struct i2c_client *client)
443 {
444     struct iio_dev *indio_dev;
445     struct mma8452_data *data;
446     int ret;
447
448     ret = i2c_smbus_read_byte_data(client, MMA8452_WHO_AM_I);
449     if (ret < 0) {
450         dev_err(&client->dev, "WHO_AM_I read failed: %d\n", ret);
451         return ret;
452     }
453
454     if (ret != MMA8452_WHO_AM_I_VAL) {
455         dev_err(&client->dev, "unexpected WHO_AM_I = 0x%02x\n", ret);
456         return -ENODEV;
457     }
458
459     indio_dev = devm_iio_device_alloc(&client->dev, sizeof(*data));
460     if (!indio_dev)
461         return -ENOMEM;
462
463     data = iio_priv(indio_dev);
464     data->client = client;
465     mutex_init(&data->lock);
466
467     i2c_set_clientdata(client, indio_dev);
468
469     indio_dev->name = "mma8452q";
470     indio_dev->info = &mma8452_iio_info;
471     indio_dev->modes = INDIO_DIRECT_MODE | INDIO_BUFFER_TRIGGERED;
472     indio_dev->channels = mma8452_channels;
473     indio_dev->num_channels = ARRAY_SIZE(mma8452_channels);
474     indio_dev->available_scan_masks = mma8452_scan_masks;
475
476     ret = mma8452_chip_init(data);
477     if (ret < 0) {
478         dev_err(&client->dev, "chip init failed: %d\n", ret);
479         return ret;
480     }
481
482     data->trig = devm_iio_trigger_alloc(&client->dev,
483                                       "%s-trigger",
484                                       indio_dev->name);
485     if (!data->trig)
486         return -ENOMEM;
487
488     data->trig->ops = &mma8452_trigger_ops;
489     iio_trigger_set_drvdata(data->trig, indio_dev);
490
491     ret = devm_iio_trigger_register(&client->dev, data->trig);
492     if (ret < 0) {
493         dev_err(&client->dev, "failed to register trigger: %d\n", ret);
494         return ret;
495     }
496
497     indio_dev->trig = iio_trigger_get(data->trig);
498
499     ret = devm_iio_triggered_buffer_setup(&client->dev,
500                                         indio_dev,
501                                         iio_pollfunc_store_time,
502                                         mma8452_trigger_handler,
503                                         NULL);
504     if (ret < 0) {
505         iio_trigger_put(indio_dev->trig);
506         indio_dev->trig = NULL;
507         dev_err(&client->dev, "triggered buffer setup failed: %d\n", ret);
508         return ret;

```

```

509 }
510
511 dev_info(&client->dev, "client irq = %d\n", client->irq);
512
513 if (client->irq > 0) {
514     ret = devm_request_threaded_irq(&client->dev,
515         client->irq,
516         NULL,
517         mma8452_irq_thread,
518         IRQF_ONESHOT,
519         "mma8452q",
520         indio_dev);
521     if (ret < 0) {
522         iio_trigger_put(indio_dev->trig);
523         indio_dev->trig = NULL;
524         dev_err(&client->dev, "failed to request irq: %d\n", ret);
525         return ret;
526     }
527
528     dev_info(&client->dev, "irq registered: %d\n", client->irq);
529 } else {
530     dev_info(&client->dev, "no irq configured\n");
531 }
532
533 ret = devm_iio_device_register(&client->dev, indio_dev);
534 if (ret < 0) {
535     iio_trigger_put(indio_dev->trig);
536     indio_dev->trig = NULL;
537     dev_err(&client->dev, "iio registration failed: %d\n", ret);
538     return ret;
539 }
540
541 dev_info(&client->dev,
542     "MMA8452Q IIO registered with interrupt trigger and buffer\n");
543
544 return 0;
545 }
546
547 static void mma8452_remove(struct i2c_client *client)
548 {
549     struct iio_dev *indio_dev = i2c_get_clientdata(client);
550     struct mma8452_data *data;
551
552     if (!indio_dev)
553         return;
554
555     data = iio_priv(indio_dev);
556
557     mma8452_set_active(data, false);
558
559     if (indio_dev->trig) {
560         iio_trigger_put(indio_dev->trig);
561         indio_dev->trig = NULL;
562     }
563
564     dev_info(&client->dev, "removed\n");
565 }
566
567 static const struct of_device_id mma8452_of_match[] = {
568     { .compatible = "nxp,mma8452q" },
569     { }
570 };
571 MODULE_DEVICE_TABLE(of, mma8452_of_match);
572
573 static const struct i2c_device_id mma8452_id[] = {
574     { "mma8452q", 0 },
575     { }
576 };
577 MODULE_DEVICE_TABLE(i2c, mma8452_id);
578
579 static struct i2c_driver mma8452_driver = {
580     .driver = {
581         .name = "mma8452q",
582         .of_match_table = mma8452_of_match,
583     },
584     .probe = mma8452_probe,

```

```

585     .remove = mma8452_remove,
586     .id_table = mma8452_id,
587 };
588
589 module_i2c_driver(mma8452_driver);
590
591 MODULE_LICENSE("GPL");
592 MODULE_AUTHOR("Andronie Vasile-Laurentiu");
593 MODULE_DESCRIPTION("MMA8452Q IIO accelerometer driver "
594                    "with interrupt trigger and buffer");

```

.2 Aplicația user-space Python

```

1  #!/usr/bin/env python3
2  """
3  mma8452q_gui_v3.py
4
5  Aplicatie grafica user-space pentru driverul Linux IIO MMA8452Q.
6
7  Functii:
8      - detectare automata /sys/bus/iio/devices/iio:deviceX dupa name == mma8452q
9      - afisare raw X/Y/Z
10     - conversie raw -> g folosind in_accel_scale
11     - calcul modul vector acceleratie |a|
12     - setare range: 2g / 4g / 8g
13     - setare sampling frequency: 1.56, 6.25, 12.5, 50, 100, 200, 400, 800 Hz
14     - afisare live in direct mode
15     - calibrare offset din N esantioane
16     - logging CSV in direct mode
17     - citire buffer IIO din /dev/iio:deviceX
18     - masurare rata reala in buffer mode
19     - cleanup buffer
20
21  Rulare:
22      python3 mma8452q_user.py
23      sudo python3 mma8452q_user.py      # necesar doar pentru buffer mode daca /dev/iio:deviceX
24      cere root
25
26  """
27
28  from __future__ import annotations
29
30  import csv
31  import math
32  import os
33  import queue
34  import struct
35  import threading
36  import time
37  import tkinter as tk
38  from dataclasses import dataclass
39  from pathlib import Path
40  from tkinter import filedialog, messagebox, ttk
41  from typing import Optional, TextIO
42
43  IIO_ROOT = Path("/sys/bus/iio/devices")
44  DEFAULT_DEVICE_NAME = "mma8452q"
45
46  RANGE_TO_SCALE = {
47      "2g": "0.000976562",
48      "4g": "0.001953125",
49      "8g": "0.003906250",
50  }
51
52  VALID_FREQ_STRINGS = {
53      "1.56": "1.560000",
54      "6.25": "6.250000",
55      "12.5": "12.500000",
56      "50": "50",
57      "100": "100",
58      "200": "200",
59      "400": "400",
60      "800": "800",
61  }

```

```

60
61 FREQ_DISPLAY = ["1.56", "6.25", "12.5", "50", "100", "200", "400", "800"]
62
63 DEFAULT_RANGE = "2g"
64 DEFAULT_FREQ = "100"
65 BUFFER_UI_PERIOD_S = 0.08
66 BUFFER_RATE_PERIOD_S = 1.00
67
68
69 class IIIOError(RuntimeError):
70     pass
71
72
73 @dataclass
74 class Offsets:
75     x: float = 0.0
76     y: float = 0.0
77     z: float = 0.0
78
79
80 @dataclass
81 class Sample:
82     timestamp_user_ns: int
83     x_raw: int
84     y_raw: int
85     z_raw: int
86     x_g: float
87     y_g: float
88     z_g: float
89     magnitude_g: float
90     scale: float
91     sampling_frequency: float
92     timestamp_iio_ns: Optional[int] = None
93
94
95 class MMA8452QDevice:
96     def __init__(self, device_name: str = DEFAULT_DEVICE_NAME, device_path: Optional[Path] =
None):
97         self.device_name = device_name
98         self.dev_path = device_path if device_path else self.find_iio_device(device_name)
99         self.dev_num = self.dev_path.name.replace("iio:device", "")
100         self.chardev_path = Path(f"/dev/iio:device{self.dev_num}")
101         self.offsets = Offsets()
102
103     @staticmethod
104     def find_iio_device(device_name: str = DEFAULT_DEVICE_NAME) -> Path:
105         if not IIO_ROOT.exists():
106             raise IIIOError(f"Nu exista {IIO_ROOT}. Driverul IIO nu pare incarcat.")
107
108         for dev in sorted(IIO_ROOT.glob("iio:device*")):
109             name_file = dev / "name"
110             if name_file.exists():
111                 try:
112                     if name_file.read_text().strip() == device_name:
113                         return dev
114                 except PermissionError:
115                     continue
116
117         found = []
118         for dev in sorted(IIO_ROOT.glob("iio:device*")):
119             name_file = dev / "name"
120             if name_file.exists():
121                 try:
122                     found.append(f"{dev.name}: {name_file.read_text().strip()}")
123                 except Exception:
124                     found.append(f"{dev.name}: <nu pot citi>")
125         raise IIIOError(f"Nu am gasit IIO device cu name == {device_name}. Gasite: {'', '.join(
found) or 'nimic'}")
126
127     def p(self, relative: str) -> Path:
128         return self.dev_path / relative
129
130     @staticmethod
131     def read_text(path: Path) -> str:
132         try:
133             return path.read_text().strip()

```

```

134     except FileNotFoundError as exc:
135         raise IOError(f"Fisier lipsa: {path}") from exc
136     except PermissionError as exc:
137         raise IOError(f"Nu ai permisiune de citire: {path}") from exc
138
139 @staticmethod
140 def write_text(path: Path, value: str) -> None:
141     try:
142         path.write_text(str(value))
143     except FileNotFoundError as exc:
144         raise IOError(f"Fisier lipsa: {path}") from exc
145     except PermissionError as exc:
146         raise IOError(f"Nu ai permisiune de scriere: {path}. Ruleaza cu sudo sau schimba
147         permisiunile.") from exc
148     except OSError as exc:
149         raise IOError(f"Nu pot scrie '{value}' in {path}: {exc}") from exc
150
151 def read_raw_xyz(self) -> tuple[int, int, int]:
152     return (
153         int(self.read_text(self.p("in_accel_x_raw"))),
154         int(self.read_text(self.p("in_accel_y_raw"))),
155         int(self.read_text(self.p("in_accel_z_raw"))),
156     )
157
158 def read_scale(self) -> float:
159     return float(self.read_text(self.p("in_accel_scale")))
160
161 def read_scale_available(self) -> str:
162     return self.read_text(self.p("in_accel_scale_available"))
163
164 def read_sampling_frequency(self) -> float:
165     return float(self.read_text(self.p("in_accel_sampling_frequency")))
166
167 def read_sampling_frequency_available(self) -> str:
168     return self.read_text(self.p("in_accel_sampling_frequency_available"))
169
170 def set_range(self, range_name: str) -> None:
171     if range_name not in RANGE_TO_SCALE:
172         raise IOError("Range invalid. Alege 2g, 4g sau 8g.")
173     self.write_text(self.p("in_accel_scale"), RANGE_TO_SCALE[range_name])
174
175 def set_sampling_frequency(self, freq: str) -> None:
176     if freq not in VALID_FREQ_STRINGS:
177         raise IOError("Frecventa invalida.")
178     self.write_text(self.p("in_accel_sampling_frequency"), VALID_FREQ_STRINGS[freq])
179
180 def read_sample(self) -> Sample:
181     scale = self.read_scale()
182     freq = self.read_sampling_frequency()
183     x_raw, y_raw, z_raw = self.read_raw_xyz()
184
185     x_cal = x_raw - self.offsets.x
186     y_cal = y_raw - self.offsets.y
187     z_cal = z_raw - self.offsets.z
188
189     x_g = x_cal * scale
190     y_g = y_cal * scale
191     z_g = z_cal * scale
192     mag = math.sqrt(x_g * x_g + y_g * y_g + z_g * z_g)
193
194     return Sample(time.time_ns(), x_raw, y_raw, z_raw, x_g, y_g, z_g, mag, scale, freq)
195
196 def calibrate(self, n: int, delay_s: float = 0.02, stop_event: Optional[threading.Event] =
197     None) -> Offsets:
198     if n <= 0:
199         raise IOError("Numarul de esantioane trebuie sa fie pozitiv.")
200     sx = sy = sz = 0.0
201     count = 0
202     for _ in range(n):
203         if stop_event and stop_event.is_set():
204             break
205         x, y, z = self.read_raw_xyz()
206         sx += x
207         sy += y
208         sz += z
209         count += 1

```



```

208         time.sleep(delay_s)
209     if count == 0:
210         raise IIIOError("Calibrare oprita inainte de primul esantion.")
211     self.offsets = Offsets(sx / count, sy / count, sz / count)
212     return self.offsets
213
214     @staticmethod
215     def csv_header() -> list[str]:
216         return [
217             "timestamp_user_ns", "timestamp_iio_ns",
218             "x_raw", "y_raw", "z_raw",
219             "x_g", "y_g", "z_g", "magnitude_g",
220             "scale", "sampling_frequency",
221         ]
222
223     @staticmethod
224     def csv_row(sample: Sample) -> list[object]:
225         return [
226             sample.timestamp_user_ns,
227             sample.timestamp_iio_ns if sample.timestamp_iio_ns is not None else "",
228             sample.x_raw, sample.y_raw, sample.z_raw,
229             f"{sample.x_g:.9f}", f"{sample.y_g:.9f}", f"{sample.z_g:.9f}",
230             f"{sample.magnitude_g:.9f}",
231             f"{sample.scale:.9f}", f"{sample.sampling_frequency:.6f}",
232         ]
233
234     def cleanup_buffer(self) -> None:
235         for rel, value in [
236             ("buffer/enable", "0"),
237             ("trigger/current_trigger", ""),
238             ("scan_elements/in_accel_x_en", "0"),
239             ("scan_elements/in_accel_y_en", "0"),
240             ("scan_elements/in_accel_z_en", "0"),
241             ("scan_elements/in_timestamp_en", "0"),
242         ]:
243             path = self.p(rel)
244             if path.exists():
245                 try:
246                     self.write_text(path, value)
247                 except IIIOError:
248                     pass
249
250     def find_trigger_name(self) -> str:
251         own = f"{self.device_name}-trigger"
252         names = []
253         for trig in sorted(IIIO_ROOT.glob("trigger*")):
254             name_file = trig / "name"
255             if name_file.exists():
256                 name = self.read_text(name_file)
257                 names.append(name)
258                 if name == own:
259                     return name
260
261         if names:
262             return names[0]
263         raise IIIOError("Nu exista trigger IIO disponibil.")
264
265     def setup_buffer(self, length: int, trigger_name: Optional[str] = None) -> None:
266         enable = self.p("buffer/enable")
267         if not enable.exists():
268             raise IIIOError("Driverul nu expune buffer/enable.")
269         if self.read_text(enable) == "1":
270             raise IIIOError("Bufferul este deja activ. Apasa Cleanup Buffer.")
271
272         trigger_name = trigger_name or self.find_trigger_name()
273
274         for channel in ["in_accel_x_en", "in_accel_y_en", "in_accel_z_en", "in_timestamp_en"]:
275             path = self.p(f"scan_elements/{channel}")
276             if path.exists():
277                 self.write_text(path, "1")
278
279         self.write_text(self.p("buffer/length"), str(length))
280         self.write_text(self.p("trigger/current_trigger"), trigger_name)
281         self.write_text(enable, "1")
282
283     @staticmethod
284     def sign_extend(value: int, bits: int) -> int:

```

```

284         sign_bit = 1 << (bits - 1)
285         return (value ^ sign_bit) - sign_bit
286
287     @classmethod
288     def decode_buffer_sample(cls, data: bytes) -> tuple[int, int, int, int]:
289         if len(data) < 16:
290             raise IOError("Sample buffer incomplet.")
291         x16, y16, z16 = struct.unpack_from("<hhh", data, 0)
292         timestamp_ns = struct.unpack_from("<q", data, 8)[0]
293
294         def norm(v: int) -> int:
295             return cls.sign_extend(v & 0xFFFF, 12)
296
297         return norm(x16), norm(y16), norm(z16), timestamp_ns
298
299     def make_sample_from_raw(self, x_raw: int, y_raw: int, z_raw: int, timestamp_iio_ns:
Optional[int] = None, scale: Optional[float] = None, freq: Optional[float] = None) ->
Sample:
300         scale = self.read_scale() if scale is None else scale
301         freq = self.read_sampling_frequency() if freq is None else freq
302
303         x_cal = x_raw - self.offsets.x
304         y_cal = y_raw - self.offsets.y
305         z_cal = z_raw - self.offsets.z
306
307         x_g = x_cal * scale
308         y_g = y_cal * scale
309         z_g = z_cal * scale
310         mag = math.sqrt(x_g * x_g + y_g * y_g + z_g * z_g)
311         return Sample(time.time_ns(), x_raw, y_raw, z_raw, x_g, y_g, z_g, mag, scale, freq,
timestamp_iio_ns)
312
313     def read_one_buffer_sample(self) -> Sample:
314         if not self.chardev_path.exists():
315             raise IOError(f"Nu exista {self.chardev_path}")
316         scale = self.read_scale()
317         freq = self.read_sampling_frequency()
318         with self.chardev_path.open("rb", buffering=0) as f:
319             raw = f.read(16)
320         x_raw, y_raw, z_raw, ts_iio = self.decode_buffer_sample(raw)
321         return self.make_sample_from_raw(x_raw, y_raw, z_raw, ts_iio, scale, freq)
322
323
324 class MMA8452QGUI(tk.Tk):
325     def __init__(self):
326         super().__init__()
327         self.title("MMA8452Q IIO Monitor")
328         self.geometry("1250x850")
329         self.minsize(1150, 780)
330
331         self.dev: Optional[MMA8452QDevice] = None
332         self.worker_stop = threading.Event()
333         self.worker_thread: Optional[threading.Thread] = None
334         self.ui_queue: queue.Queue = queue.Queue()
335
336         self.csv_file: Optional[TextIO] = None
337         self.csv_writer: Optional[csv.writer] = None
338         self.log_path: Optional[Path] = None
339         self.sample_history: list[Sample] = []
340         self.max_history = 80
341         self.live_running = False
342         self.buffer_running = False
343         self.buffer_start_time = 0.0
344         self.buffer_count = 0
345         self.buffer_last_ui_time = 0.0
346         self.buffer_ui_period_s = BUFFER_UI_PERIOD_S
347
348         self._build_ui()
349         self.after(33, self.process_queue)
350         self.connect_device(show_errors=False)
351         self.protocol("WM_DELETE_WINDOW", self.on_close)
352
353     def _build_ui(self) -> None:
354         self.columnconfigure(0, weight=1)
355         self.rowconfigure(2, weight=1)
356

```

```

357         top = ttk.LabelFrame(self, text="Device")
358         top.grid(row=0, column=0, sticky="ew", padx=10, pady=8)
359         top.columnconfigure(1, weight=1)
360
361         self.device_var = tk.StringVar(value="-")
362         self.chardev_var = tk.StringVar(value="-")
363         self.status_var = tk.StringVar(value="Neconectat")
364
365         ttk.Label(top, text="IIO:").grid(row=0, column=0, sticky="w", padx=6, pady=3)
366         ttk.Label(top, textvariable=self.device_var).grid(row=0, column=1, sticky="ew", padx
=6, pady=3)
367         ttk.Label(top, text="Char:").grid(row=1, column=0, sticky="w", padx=6, pady=3)
368         ttk.Label(top, textvariable=self.chardev_var).grid(row=1, column=1, sticky="ew", padx
=6, pady=3)
369         ttk.Label(top, text="Status:").grid(row=2, column=0, sticky="w", padx=6, pady=3)
370         ttk.Label(top, textvariable=self.status_var).grid(row=2, column=1, sticky="ew", padx
=6, pady=3)
371         ttk.Button(top, text="Detecteaza", command=self.connect_device).grid(row=0, column=2,
rowspan=2, padx=6, pady=3, sticky="ns")
372         ttk.Button(top, text="Cleanup Buffer", command=self.cleanup_buffer).grid(row=2, column
=2, padx=6, pady=3, sticky="ew")
373
374         controls = ttk.LabelFrame(self, text="Configurare")
375         controls.grid(row=1, column=0, sticky="ew", padx=10, pady=4)
376         for i in range(8):
377             controls.columnconfigure(i, weight=1)
378
379         self.range_var = tk.StringVar(value="2g")
380         self.freq_var = tk.StringVar(value="100")
381         self.period_var = tk.StringVar(value="0.2")
382         self.calib_var = tk.StringVar(value="100")
383         self.buffer_len_var = tk.StringVar(value="64")
384
385         ttk.Label(controls, text="Range").grid(row=0, column=0, padx=5, pady=4)
386         ttk.Combobox(controls, textvariable=self.range_var, values=["2g", "4g", "8g"], width
=8, state="readonly").grid(row=1, column=0, padx=5, pady=4)
387         ttk.Button(controls, text="Set range", command=self.set_range).grid(row=2, column=0,
padx=5, pady=4)
388
389         ttk.Label(controls, text="Frecventa [Hz]").grid(row=0, column=1, padx=5, pady=4)
390         ttk.Combobox(controls, textvariable=self.freq_var, values=FREQ_DISPLAY, width=8, state
="readonly").grid(row=1, column=1, padx=5, pady=4)
391         ttk.Button(controls, text="Set freq", command=self.set_frequency).grid(row=2, column
=1, padx=5, pady=4)
392
393         ttk.Label(controls, text="Perioada live [s]").grid(row=0, column=2, padx=5, pady=4)
394         ttk.Entry(controls, textvariable=self.period_var, width=8).grid(row=1, column=2, padx
=5, pady=4)
395         ttk.Button(controls, text="Start Live", command=self.start_live).grid(row=2, column=2,
padx=5, pady=4)
396
397         ttk.Label(controls, text="Calibrare N").grid(row=0, column=3, padx=5, pady=4)
398         ttk.Entry(controls, textvariable=self.calib_var, width=8).grid(row=1, column=3, padx
=5, pady=4)
399         ttk.Button(controls, text="Calibreaza", command=self.start_calibration).grid(row=2,
column=3, padx=5, pady=4)
400
401         ttk.Label(controls, text="Buffer length").grid(row=0, column=4, padx=5, pady=4)
402         ttk.Entry(controls, textvariable=self.buffer_len_var, width=8).grid(row=1, column=4,
padx=5, pady=4)
403         ttk.Button(controls, text="Start Buffer", command=self.start_buffer).grid(row=2,
column=4, padx=5, pady=4)
404
405         ttk.Button(controls, text="Citeste o data", command=self.read_once).grid(row=1, column
=5, padx=5, pady=4, sticky="ew")
406         ttk.Button(controls, text="Stop", command=self.stop_worker).grid(row=2, column=5, padx
=5, pady=4, sticky="ew")
407         ttk.Button(controls, text="Alege CSV", command=self.choose_csv).grid(row=1, column=6,
padx=5, pady=4, sticky="ew")
408         ttk.Button(controls, text="Opreste CSV", command=self.close_csv).grid(row=2, column=6,
padx=5, pady=4, sticky="ew")
409
410         self.logging_var = tk.StringVar(value="CSV: oprit")
411         ttk.Label(controls, textvariable=self.logging_var).grid(row=1, column=7, rowspan=2,
padx=5, pady=4, sticky="w")
412

```

```

413     main = ttk.Frame(self)
414     main.grid(row=2, column=0, sticky="nsew", padx=10, pady=8)
415     main.columnconfigure(0, weight=1)
416     main.columnconfigure(1, weight=1)
417     main.rowconfigure(0, weight=1)
418
419     values = ttk.LabelFrame(main, text="Valori curente")
420     values.grid(row=0, column=0, sticky="nsew", padx=(0, 6))
421     for i in range(4):
422         values.columnconfigure(i, weight=1)
423
424     self.raw_vars = {axis: tk.StringVar(value="0") for axis in "XYZ"}
425     self.g_vars = {axis: tk.StringVar(value="0.00000") for axis in "XYZ"}
426     self.mag_var = tk.StringVar(value="0.00000")
427     self.scale_var = tk.StringVar(value="-")
428     self.sampling_var = tk.StringVar(value="-")
429     self.offset_var = tk.StringVar(value="X=0, Y=0, Z=0")
430     self.rate_var = tk.StringVar(value="Rata buffer: -")
431
432     ttk.Label(values, text="Axa", font=("TkDefaultFont", 10, "bold")).grid(row=0, column
=0, pady=4)
433     ttk.Label(values, text="Raw", font=("TkDefaultFont", 10, "bold")).grid(row=0, column
=1, pady=4)
434     ttk.Label(values, text="g", font=("TkDefaultFont", 10, "bold")).grid(row=0, column=2,
pady=4)
435
436     for idx, axis in enumerate("XYZ", start=1):
437         ttk.Label(values, text=axis, font=("TkDefaultFont", 13, "bold")).grid(row=idx,
column=0, pady=6)
438         ttk.Label(values, textvariable=self.raw_vars[axis], font=("TkDefaultFont", 13)).
grid(row=idx, column=1, pady=6)
439         ttk.Label(values, textvariable=self.g_vars[axis], font=("TkDefaultFont", 13)).grid
(row=idx, column=2, pady=6)
440
441     ttk.Separator(values).grid(row=4, column=0, columnspan=3, sticky="ew", pady=8)
442     ttk.Label(values, text="|a| [g]").grid(row=5, column=0, sticky="e", padx=6, pady=4)
443     ttk.Label(values, textvariable=self.mag_var, font=("TkDefaultFont", 14, "bold")).grid(
row=5, column=1, columnspan=2, sticky="w", padx=6, pady=4)
444     ttk.Label(values, text="Scale").grid(row=6, column=0, sticky="e", padx=6, pady=4)
445     ttk.Label(values, textvariable=self.scale_var).grid(row=6, column=1, columnspan=2,
sticky="w", padx=6, pady=4)
446     ttk.Label(values, text="Sampling").grid(row=7, column=0, sticky="e", padx=6, pady=4)
447     ttk.Label(values, textvariable=self.sampling_var).grid(row=7, column=1, columnspan=2,
sticky="w", padx=6, pady=4)
448     ttk.Label(values, text="Offset raw").grid(row=8, column=0, sticky="e", padx=6, pady=4)
449     ttk.Label(values, textvariable=self.offset_var).grid(row=8, column=1, columnspan=2,
sticky="w", padx=6, pady=4)
450     ttk.Label(values, textvariable=self.rate_var).grid(row=9, column=0, columnspan=3,
sticky="w", padx=6, pady=4)
451
452     graph_box = ttk.LabelFrame(main, text="Grafic simplu |a| [g]")
453     graph_box.grid(row=0, column=1, sticky="nsew", padx=(6, 0))
454     graph_box.rowconfigure(0, weight=1)
455     graph_box.columnconfigure(0, weight=1)
456     self.canvas = tk.Canvas(graph_box, background="white", height=260)
457     self.canvas.grid(row=0, column=0, sticky="nsew", padx=8, pady=8)
458
459     bottom = ttk.LabelFrame(self, text="Log evenimente")
460     bottom.grid(row=3, column=0, sticky="nsew", padx=10, pady=(0, 10))
461     bottom.columnconfigure(0, weight=1)
462     bottom.rowconfigure(0, weight=1)
463     self.log_text = tk.Text(bottom, height=8)
464     self.log_text.grid(row=0, column=0, sticky="nsew")
465     scroll = ttk.Scrollbar(bottom, orient="vertical", command=self.log_text.yview)
466     scroll.grid(row=0, column=1, sticky="ns")
467     self.log_text.configure(yscrollcommand=scroll.set)
468
469     def log(self, msg: str) -> None:
470         ts = time.strftime("%H:%M:%S")
471         self.log_text.insert("end", f"[{ts}] {msg}\n")
472         self.log_text.see("end")
473
474     def connect_device(self, show_errors: bool = True) -> None:
475         try:
476             self.dev = MMA8452QDevice()
477             self.device_var.set(str(self.dev.dev_path))

```

```

478         self.chardev_var.set(str(self.dev.chardev_path))
479         self.status_var.set("Conectat")
480         self.log("Device detectat corect.")
481         self.apply_defaults_on_startup()
482         self.refresh_static_info()
483     except Exception as exc:
484         self.dev = None
485         self.status_var.set("Eroare detectare")
486         if show_errors:
487             messagebox.showerror("Eroare", str(exc))
488         self.log(str(exc))
489
490 def apply_defaults_on_startup(self) -> None:
491     """
492     La pornirea aplicatiei scriem explicit configuratia implicita in driver.
493     Altfel, sysfs pastreaza starea ramasa din testul anterior, de exemplu 800 Hz.
494     """
495     try:
496         dev = self.require_dev()
497
498         dev.cleanup_buffer()
499         dev.set_range(DEFAULT_RANGE)
500         dev.set_sampling_frequency(DEFAULT_FREQ)
501         self.range_var.set(DEFAULT_RANGE)
502         self.freq_var.set(DEFAULT_FREQ)
503         self.log(f"Default aplicat la pornire: range={DEFAULT_RANGE}, freq={DEFAULT_FREQ}
504 Hz.")
505     except Exception as exc:
506         self.log(f"Nu am putut aplica default la pornire: {exc}")
507
508 def require_dev(self) -> MMA8452QDevice:
509     if self.dev is None:
510         raise IOError("Device-ul nu este detectat.")
511     return self.dev
512
513 def refresh_static_info(self) -> None:
514     dev = self.require_dev()
515     self.scale_var.set(f"{dev.read_scale():.9f}")
516     self.sampling_var.set(f"{dev.read_sampling_frequency():g} Hz")
517     self.offset_var.set(f"X={dev.offsets.x:.2f}, Y={dev.offsets.y:.2f}, Z={dev.offsets.z:.2f}")
518
519 def set_range(self) -> None:
520     try:
521         dev = self.require_dev()
522         dev.set_range(self.range_var.get())
523         self.refresh_static_info()
524         self.log(f"Range setat la {self.range_var.get()}")
525     except Exception as exc:
526         messagebox.showerror("Eroare", str(exc))
527         self.log(str(exc))
528
529 def set_frequency(self) -> None:
530     try:
531         dev = self.require_dev()
532         dev.set_sampling_frequency(self.freq_var.get())
533         self.refresh_static_info()
534         self.log(f"Frecventa ceruta: {self.freq_var.get()} Hz; frecventa citita din driver
535 : {dev.read_sampling_frequency():g} Hz.")
536     except Exception as exc:
537         messagebox.showerror("Eroare", str(exc))
538         self.log(str(exc))
539
540 def update_sample_ui(self, sample: Sample) -> None:
541     self.raw_vars["X"].set(str(sample.x_raw))
542     self.raw_vars["Y"].set(str(sample.y_raw))
543     self.raw_vars["Z"].set(str(sample.z_raw))
544     self.g_vars["X"].set(f"{sample.x_g:+.5f}")
545     self.g_vars["Y"].set(f"{sample.y_g:+.5f}")
546     self.g_vars["Z"].set(f"{sample.z_g:+.5f}")
547     self.mag_var.set(f"{sample.magnitude_g:.5f}")
548     self.scale_var.set(f"{sample.scale:.9f}")
549     self.sampling_var.set(f"{sample.sampling_frequency:g} Hz")
550
551     self.sample_history.append(sample)
552     if len(self.sample_history) > self.max_history:

```

```

551         self.sample_history = self.sample_history[-self.max_history:]
552     self.draw_graph()
553     self.write_csv_sample(sample)
554
555     def draw_graph(self) -> None:
556         self.canvas.delete("all")
557         w = max(self.canvas.winfo_width(), 10)
558         h = max(self.canvas.winfo_height(), 10)
559         pad = 28
560
561         range_name = self.range_var.get()
562         graph_max_g = {
563             "2g": 2.0,
564             "4g": 4.0,
565             "8g": 8.0,
566         }.get(range_name, 8.0)
567
568         mid_g = graph_max_g / 2.0
569
570         self.canvas.create_line(pad, h - pad, w - pad, h - pad)
571         self.canvas.create_line(pad, pad, pad, h - pad)
572
573         self.canvas.create_text(8, pad, anchor="w", text=f"{graph_max_g:g}g")
574         self.canvas.create_text(8, h // 2, anchor="w", text=f"{mid_g:g}g")
575         self.canvas.create_text(8, h - pad, anchor="w", text="0g")
576
577         if len(self.sample_history) < 2:
578             return
579
580         mags = [min(max(s.magnitude_g, 0.0), graph_max_g) for s in self.sample_history]
581         window = 4
582         if len(mags) >= window:
583             mags = [
584                 sum(mags[max(0, i - window + 1):i + 1]) / len(mags[max(0, i - window + 1):i +
1])
585                 for i in range(len(mags))
586             ]
587         step = (w - 2 * pad) / max(len(mags) - 1, 1)
588
589         pts = []
590         for i, mag in enumerate(mags):
591             x = pad + i * step
592             y = h - pad - (mag / graph_max_g) * (h - 2 * pad)
593             pts.extend([x, y])
594
595         self.canvas.create_line(*pts, width=2)
596
597
598         if graph_max_g >= 1.0:
599             y_1g = h - pad - (1.0 / graph_max_g) * (h - 2 * pad)
600             self.canvas.create_line(pad, y_1g, w - pad, y_1g, dash=(4, 3))
601             self.canvas.create_text(w - pad - 20, y_1g - 8, text="1g")
602
603     def read_once(self) -> None:
604         try:
605             sample = self.require_dev().read_sample()
606             self.update_sample_ui(sample)
607             self.log("Citire directa executata.")
608         except Exception as exc:
609             messagebox.showerror("Eroare", str(exc))
610             self.log(str(exc))
611
612     def choose_csv(self) -> None:
613         path = filedialog.asksaveasfilename(
614             title="Alege fisier CSV",
615             defaultextension=".csv",
616             filetypes=[("CSV", "*.csv"), ("All files", "*.*")],
617         )
618         if not path:
619             return
620         try:
621             self.close_csv()
622             self.log_path = Path(path)
623             self.csv_file = self.log_path.open("w", newline="")
624             self.csv_writer = csv.writer(self.csv_file)
625             self.csv_writer.writerow(MMA8452QDevice.csv_header())

```

```

626         self.logging_var.set(f"CSV: {self.log_path.name}")
627         self.log(f"Logging CSV pornit: {self.log_path}")
628     except Exception as exc:
629         messagebox.showerror("Eroare CSV", str(exc))
630         self.log(str(exc))
631
632     def close_csv(self) -> None:
633         if self.csv_file:
634             self.csv_file.close()
635         self.csv_file = None
636         self.csv_writer = None
637         self.log_path = None
638         self.logging_var.set("CSV: oprit")
639
640     def write_csv_sample(self, sample: Sample) -> None:
641         if self.csv_writer and self.csv_file:
642             self.csv_writer.writerow(MMA8452QDevice.csv_row(sample))
643             self.csv_file.flush()
644
645     def stop_worker(self) -> None:
646         self.worker_stop.set()
647         self.live_running = False
648         self.buffer_running = False
649         self.status_var.set("Oprire ceruta")
650         self.log("Oprire ceruta.")
651
652     def start_live(self) -> None:
653         if self.worker_thread and self.worker_thread.is_alive():
654             messagebox.showwarning("Atentie", "Exista deja o operatie pornita.")
655             return
656         try:
657             period = float(self.period_var.get())
658             if period <= 0:
659                 raise ValueError
660         except ValueError:
661             messagebox.showerror("Eroare", "Perioada trebuie sa fie numar pozitiv.")
662             return
663
664         self.worker_stop.clear()
665         self.live_running = True
666         self.status_var.set("Live direct mode")
667         self.worker_thread = threading.Thread(target=self.live_worker, args=(period,), daemon=
True)
668         self.worker_thread.start()
669         self.log("Live direct mode pornit.")
670
671     def live_worker(self, period: float) -> None:
672         try:
673             dev = self.require_dev()
674             while not self.worker_stop.is_set():
675                 sample = dev.read_sample()
676                 self.ui_queue.put(("sample", sample))
677                 time.sleep(period)
678         except Exception as exc:
679             self.ui_queue.put(("error", str(exc)))
680         finally:
681             self.ui_queue.put(("status", "Live oprit"))
682
683     def start_calibration(self) -> None:
684         if self.worker_thread and self.worker_thread.is_alive():
685             messagebox.showwarning("Atentie", "Opreste live/buffer inainte de calibrare.")
686             return
687         try:
688             n = int(self.calib_var.get())
689             if n <= 0:
690                 raise ValueError
691         except ValueError:
692             messagebox.showerror("Eroare", "N trebuie sa fie intreg pozitiv.")
693             return
694         self.worker_stop.clear()
695         self.status_var.set("Calibrare...")
696         self.worker_thread = threading.Thread(target=self.calibration_worker, args=(n,),
daemon=True)
697         self.worker_thread.start()
698         self.log("Calibrare pornita. Tine senzorul nemiscat.")
699

```



```

700 def calibration_worker(self, n: int) -> None:
701     try:
702         offsets = self.require_dev().calibrate(n, stop_event=self.worker_stop)
703         self.ui_queue.put(("calibrated", offsets))
704     except Exception as exc:
705         self.ui_queue.put(("error", str(exc)))
706
707 def start_buffer(self) -> None:
708     if self.worker_thread and self.worker_thread.is_alive():
709         messagebox.showwarning("Atentie", "Exista deja o operatie pornita.")
710         return
711     try:
712         length = int(self.buffer_len_var.get())
713         if length <= 0:
714             raise ValueError
715     except ValueError:
716         messagebox.showerror("Eroare", "Buffer length trebuie sa fie intreg pozitiv.")
717         return
718
719     if self.csv_file:
720         path = self.log_path
721         self.close_csv()
722         self.log_path = path
723         if path:
724             self.logging_var.set(f"CSV buffer: {path.name}")
725
726     self.worker_stop.clear()
727     self.buffer_running = True
728     self.buffer_count = 0
729     self.buffer_start_time = time.monotonic()
730     self.buffer_last_ui_time = self.buffer_start_time
731     self.status_var.set("Buffer mode")
732     self.worker_thread = threading.Thread(target=self.buffer_worker, args=(length,),
733     daemon=True)
734     self.worker_thread.start()
735     self.log("Buffer mode pornit.")
736
737 def buffer_worker(self, length: int) -> None:
738     dev: Optional[MMA8452QDevice] = None
739     csv_file: Optional[TextIO] = None
740     csv_writer: Optional[csv.writer] = None
741     try:
742         dev = self.require_dev()
743         dev.setup_buffer(length=length)
744         self.ui_queue.put(("buffer_started", None))
745
746         if self.log_path:
747             csv_file = self.log_path.open("a", newline="")
748             csv_writer = csv.writer(csv_file)
749
750             sample_size = 16
751             scale = dev.read_scale()
752             freq = dev.read_sampling_frequency()
753             local_count = 0
754             last_ui = 0.0
755             last_rate = time.monotonic()
756             last_rate_count = 0
757
758             if not dev.chardev_path.exists():
759                 raise IOError(f"Nu exista {dev.chardev_path}")
760
761             with dev.chardev_path.open("rb", buffering=0) as f:
762                 while not self.worker_stop.is_set():
763                     raw = f.read(sample_size)
764                     if len(raw) != sample_size:
765                         continue
766
767                     x_raw, y_raw, z_raw, ts_iio = dev.decode_buffer_sample(raw)
768                     sample = dev.make_sample_from_raw(x_raw, y_raw, z_raw, ts_iio, scale, freq
769 )
770                     local_count += 1
771
772                     if csv_writer and csv_file:
773                         csv_writer.writerow(MMA8452QDevice.csv_row(sample))

```



```

774         if local_count % 64 == 0:
775             csv_file.flush()
776
777         now = time.monotonic()
778         if now - last_ui >= self.buffer_ui_period_s:
779             self.ui_queue.put(("buffer_sample", (sample, local_count, now)))
780             last_ui = now
781
782         if now - last_rate >= BUFFER_RATE_PERIOD_S:
783             real_rate = (local_count - last_rate_count) / max(now - last_rate, 1e
-9)
784
785             self.ui_queue.put(("buffer_rate", real_rate))
786             last_rate = now
787             last_rate_count = local_count
788
789         except PermissionError as exc:
790             self.ui_queue.put(("error", f"Nu ai permisiune pentru {dev.chardev_path if dev
else '/dev/iio:deviceX'}. Ruleaza cu sudo."))
791         except Exception as exc:
792             self.ui_queue.put(("error", str(exc)))
793         finally:
794             if csv_file:
795                 csv_file.flush()
796                 csv_file.close()
797             if dev:
798                 dev.cleanup_buffer()
799             self.ui_queue.put(("status", "Buffer oprit"))
800
801     def cleanup_buffer(self) -> None:
802         try:
803             self.require_dev().cleanup_buffer()
804             self.log("Buffer dezactivat si trigger curatat.")
805         except Exception as exc:
806             messagebox.showerror("Eroare", str(exc))
807             self.log(str(exc))
808
809     def process_queue(self) -> None:
810
811         max_events_per_tick = 25
812         processed = 0
813         try:
814             while processed < max_events_per_tick:
815                 kind, payload = self.ui_queue.get_nowait()
816                 processed += 1
817
818                 if kind == "sample":
819                     self.update_sample_ui(payload)
820                 elif kind == "buffer_started":
821                     self.buffer_start_time = time.monotonic()
822                     self.buffer_count = 0
823                     self.log("Buffer activat. Rata se masoara din numarul real de sample-uri
citite.")
824                 elif kind == "buffer_sample":
825                     sample, local_count, now = payload
826                     self.buffer_count = local_count
827                     elapsed = max(now - self.buffer_start_time, 1e-9)
828                     self.rate_var.set(f"Rata buffer medie: {self.buffer_count / elapsed:.2f}
samples/s")
829                     self.update_sample_ui(sample)
830                 elif kind == "buffer_rate":
831                     self.rate_var.set(f"Rata buffer reala: {payload:.2f} samples/s")
832                 elif kind == "error":
833                     self.status_var.set("Eroare")
834                     self.log(payload)
835                     messagebox.showerror("Eroare", payload)
836                     self.worker_stop.set()
837                 elif kind == "status":
838                     self.status_var.set(payload)
839                     self.log(payload)
840                 elif kind == "calibrated":
841                     off: Offsets = payload
842                     self.offset_var.set(f"X={off.x:.2f}, Y={off.y:.2f}, Z={off.z:.2f}")
843                     self.status_var.set("Calibrare finalizata")
844                     self.log(f"Offset calculat: X={off.x:.2f}, Y={off.y:.2f}, Z={off.z:.2f}")
845         except queue.Empty:

```

```

846         pass
847         self.after(100, self.process_queue)
848
849     def on_close(self) -> None:
850         self.stop_worker()
851         try:
852             if self.dev:
853                 self.dev.cleanup_buffer()
854         finally:
855             self.close_csv()
856             self.destroy()
857
858
859 if __name__ == "__main__":
860     app = MMA8452QGUI()
861     app.mainloop()

```

.3 Overlay Device Tree pentru MMA8452Q

```

1 /dts-v1/;
2 /plugin/;
3
4 / {
5     compatible = "brcm,bcm2711";
6
7     fragment@0 {
8         target = <&i2c1>;
9
10        __overlay__ {
11            #address-cells = <1>;
12            #size-cells = <0>;
13            status = "okay";
14
15            mma8452q@1c {
16                compatible = "nxp,mma8452q";
17                reg = <0x1c>;
18
19                interrupt-parent = <&gpio>;
20                interrupts = <17 8>;
21
22                status = "okay";
23            };
24        };
25    };
26 };

```

.4 Makefile pentru compilarea modulului kernel

```

1
2 obj-m += mma8452_iio.o
3
4 all:
5     make -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules
6
7 clean:
8     make -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean

```